

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики — процессов управления

Кафедра моделирования электромеханических и компьютерных систем

Шаго Павел Евгеньевич

Выпускная квалификационная работа
«Математическое моделирование и реализация
планировщика процессов для гетерогенных
процессорных архитектур»

Уровень образования: бакалавриат

Направление: 01.03.02 Прикладная математика и информатика

Основная образовательная программа СВ.5005.2022: «Прикладная математика, фундаментальная информатика и программирование»

Научный руководитель

кандидат физ.-мат. наук, доцент

Корхов Владимир Владиславович

Рецензент

Эксперт по архитектуре и сетевому стеку

ООО НИЦ АЭРОДИСК

Анохин Юрий Владимирович

Санкт-Петербург

2026

Содержание

Список сокращений	3
Введение	4
Постановка задачи	5
Глава 1. Подходы к реализации планировщиков в ОС	6
1.1. Задача планирования	6
1.2. Гетерогенные процессорные архитектуры	7
1.3. Расширяемый планировщик Linux	8
1.4. Актуальные исследования планировщиков	11
1.5. Анализ подходов к реализации планировщика	15
Глава 2. Математические модели планировщиков	18
2.1. Введение	18
2.2. Обозначения и допущения	18
2.3. Earliest Eligible Virtual Deadline First	21
2.4. Аукционная модель A1349	25
Глава 3. Реализация алгоритмов планирования	32
3.1. Реализация EEVDF с помощью sched_ext	32
3.2. Реализация аукционной модели A1349	35
Глава 4. Описание тестового окружения и экспериментального обо- рудования	48
4.1. Тестовое оборудование и программное окружение	48
4.2. Набор бенчмарков	49
4.3. Измеряемые метрики	51
Глава 5. Анализ результатов эксперимента	53
5.1. Результаты при лёгкой нагрузке	53
5.2. Результаты при умеренной нагрузке	56
5.3. Результаты при стрессовой нагрузке	58
5.4. Сводное сравнение по режимам нагрузки	61
Выводы	67
Заключение	69
Список литературы	70

Список сокращений

ARM	архитектура Advanced RISC Machine
eBPF (BPF)	расширенный фильтр пакетов Беркли (extended Berkeley Packet Filter)
CPU	центральный процессор (Central Processing Unit)
EEVDF	планировщик ближайшего допустимого виртуального дедлайна (Earliest Eligible Virtual Deadline First)
EWMA	экспоненциально взвешенное скользящее среднее (exponentially weighted moving average)
FIFO	первым пришёл, первым обслужен (First In First Out)
LAVD	планировщик с учётом критичности задержки (Latency-criticality Aware Virtual Deadline)
MDP	марковский процесс принятия решений (Markov Decision Process)
P/E	производительные / энергоэффективные ядра (Performance / Efficient)
QPS	запросов в секунду (queries per second)
RAPL	интерфейс ограничения средней мощности (Running Average Power Limit)
RISC	архитектура с сокращённым набором команд (Reduced Instruction Set Computer)
RPS	запросов в секунду (requests per second)
SCX	расширяемый класс планирования (Scheduler Class eXtensible, sched_ext)
SMT	одновременная многопоточность (Simultaneous Multithreading, Hyper-Threading)
SoC	система на кристалле (System on a Chip)
TPS	транзакций в секунду (transactions per second)
VCG	механизм Викри–Кларка–Гровза (Vickrey–Clarke–Groves)

Введение

Планировщик операционной системы (ОС) распределяет процессорное время между задачами. От качества его решений зависят пропускная способность системы под нагрузкой, время отклика интерактивных приложений и энергопотребление оборудования.

Современные процессоры всё чаще имеют гетерогенную архитектуру, при которой на одном кристалле сочетаются производительные и энергоэффективные ядра. Такая организация уже применяется в мобильных процессорах ARM big.LITTLE, в настольных и серверных процессорах Intel начиная с поколения Alder Lake, появляется и в экосистеме RISC-V. Выбор ядра при размещении задачи влияет на время её выполнения и энергопотребление, поэтому планировщик должен учитывать различие ядер.

Интерактивные, пакетные и фоновые нагрузки предъявляют к планировщику различные требования, и оптимальная для одного сценария политика редко оказывается удачной для другого. Возникает потребность подбирать политику под конкретный сценарий, однако штатный планировщик ядра Linux един для всех пользователей, а его самостоятельная модификация трудоёмка в сопровождении и плохо переносима между версиями ядра.

В ответ на эту потребность в ядро Linux был принят фреймворк `sched_ext`, позволяющий реализовывать политику планирования как программу, загружаемую в ядро во время работы системы без его пересборки. Это ускоряет разработку и снижает порог входа для прототипирования новых алгоритмов.

В настоящей работе предлагается аукционная модель планировщика A1349, учитывающая различие ядер по вычислительной мощности, как альтернатива базовому планировщику Linux. Средствами `sched_ext` реализованы обе модели, A1349 и базовая для Linux модель EEVDF. Реализация EEVDF позволяет дополнительно оценить накладные расходы фреймворка. Обе реализации сравниваются со штатным планировщиком на наборе тестов при различных типах нагрузок.

Постановка задачи

Объектом исследования являются алгоритмы планирования задач (процессов) в операционных системах на вычислительных устройствах с гетерогенной процессорной архитектурой (Intel Hybrid, ARM big.LITTLE и другие).

Предметом исследования являются методы планирования задач, учитывающие различие вычислительной мощности ядер процессора.

Целью работы является разработка и экспериментальная оценка планировщика, обеспечивающего более высокую эффективность вычислений на гетерогенном оборудовании по сравнению с базовым алгоритмом EEVDF ядра Linux.

Для достижения цели работы необходимо:

1. Проанализировать и определить наиболее перспективный подход к реализации планировщика.
2. Построить математическую модель, учитывающую различную вычислительную мощность ядер процессора.
3. Реализовать разработанный алгоритм и собрать воспроизводимый набор бенчмарков.
4. Провести эксперимент на гетерогенном оборудовании и оценить результаты в сравнении с базовым планировщиком Linux EEVDF.

Глава 1. Подходы к реализации планировщиков в ОС

1.1. Задача планирования

Операционные системы реализуют многозадачность через контекстное переключение, выделяя задачам кванты процессорного времени. Задача планирования состоит в построении расписания, минимизирующего простой оборудования и максимизирующего целевые показатели эффективности. Обычно её решает встроенный в ядро ОС компонент, называемый планировщиком, который распределяет задачи по ядрам процессора.

С развитием операционных систем и оборудования планировщики также эволюционировали. Так, в операционной системе UNIX планировщик выбирал задачу по приоритету, а среди равных задач использовался алгоритм похожий на round-robin [1]. Позднее в ядре Linux версии 2.4 был реализован планировщик $O(n)$, который при планировании следующей задачи просматривал очередь готовых к исполнению задач и вычислял для них эвристику *goodness* [2]. В Linux 2.6 его сменил планировщик $O(1)$, в котором разделили очередь приоритетов на два массива, *active* и *expired*, что позволило выбирать следующую задачу за постоянное время [3].

Затем в версии Linux 2.6.23 произошёл переход к планировщику Completely Fair Scheduler (CFS), основанному на модели виртуального времени. Готовые к выполнению задачи хранились в красно-чёрном дереве, упорядоченном по виртуальному времени, а планировщик выбирал задачу, получившую меньше всего процессорного времени относительно своей справедливой доли [4]. В Linux 6.6 эта идея получила развитие в планировщике Earliest Eligible Virtual Deadline First (EEVDF), где вводятся отставание и виртуальный дедлайн. Допустимыми считаются задачи, у которых отставание $\text{lag} \geq 0$, среди них выбирается задача с ближайшим виртуальным дедлайном [5].

Таким образом, за последние полвека планировщик операционной системы прошёл путь от детерминированных приоритетных схем, близких по логике к round-robin, до алгоритмов динамического распределения процессорного времени, основанных на серьёзных математических моделях.

1.2. Гетерогенные процессорные архитектуры

Если классические многопроцессорные системы строились на симметричном принципе (SMP), где все ядра обладают одинаковой производительностью и энергопотреблением, то современные процессоры всё чаще объединяют на одном кристалле ядра разных классов. Архитектура ARM big.LITTLE впервые сделала такой подход массовым в мобильном сегменте, объединив производительные ядра (*big*) с энергоэффективными (*LITTLE*) [6]. В настольных и серверных процессорах Intel начиная с поколения Alder Lake внедрена гибридная архитектура с Performance и Efficient ядрами (P/E) [7]. Принцип не привязан к конкретной системе команд. Аналогичные конфигурации появляются и в экосистеме RISC-V, например, гетерогенный SoC Marsellus [8].

Ядра одного процессора в таких системах отличаются по тактовой частоте, ширине внеочередного исполнения, размеру кэшей и энергопотреблению. Одна и та же задача, выполненная на P-ядре, завершается быстрее, но с большими энергозатратами, тогда как E-ядро обеспечивает лучшее соотношение производительность/ватт при более низкой пропускной способности. Размещение чувствительной к задержкам задачи на E-ядре увеличивает время её обслуживания, а выполнение фоновой нагрузки на P-ядре приводит к избыточному энергопотреблению.

В ядре Linux различия между ядрами обрабатываются подсистемой Capacity Aware Scheduling (CAS), учитывающей относительную мощность каждого ядра при балансировке. Поверх неё работает Energy Aware Scheduling (EAS), которая по модели энергопотребления и оценкам загрузки выбирает для задачи ядро с меньшим потреблением без потери пропускной способности. EAS включается лишь при наличии энергомодели, регулятора частоты schedutil и отсутствии SMT (Hyper-Threading), поэтому применяется прежде всего на мобильных SoC ARM big.LITTLE [9]. На стороне x86 для информирования планировщика о приоритете ядер применяется механизм Intel Thread Director, транслирующий аппаратные подсказки о характере нагрузки в теги класса задачи [7]. Однако CAS и Thread Director закрывают лишь балансировку по вычислительной мощности и классификацию по профилю инструкций, тогда как отдельные очереди готовых задач для P/E-кластеров, приоритизация чувствительных к задержкам

нагрузок и прикладные правила выбора кластера остаются за пределами штатной подсистемы. Реализация подобных политик требует существенных изменений ядра, что мотивирует переход к расширяемым механизмам, рассматриваемым в следующем разделе.

1.3. Расширяемый планировщик Linux

sched_ext (scheduler extensions) [10] – инфраструктура (фреймворк) ядра Linux, позволяющая реализовывать политику планирования как eBPF-программу, подключаемую к интерфейсу расширяемых планировщиков во время работы системы без пересборки ядра. sched_ext был включён в основную ветку Linux в версии 6.12 и предоставляет отдельный класс планирования `ext_sched_class`, расположенный между классами `fair_sched_class` и `idle_sched_class`. При загруженном eBPF-планировщике обычные задачи переводятся из класса справедливого планирования `fair_sched_class` (далее fair-класс) под управление sched_ext.

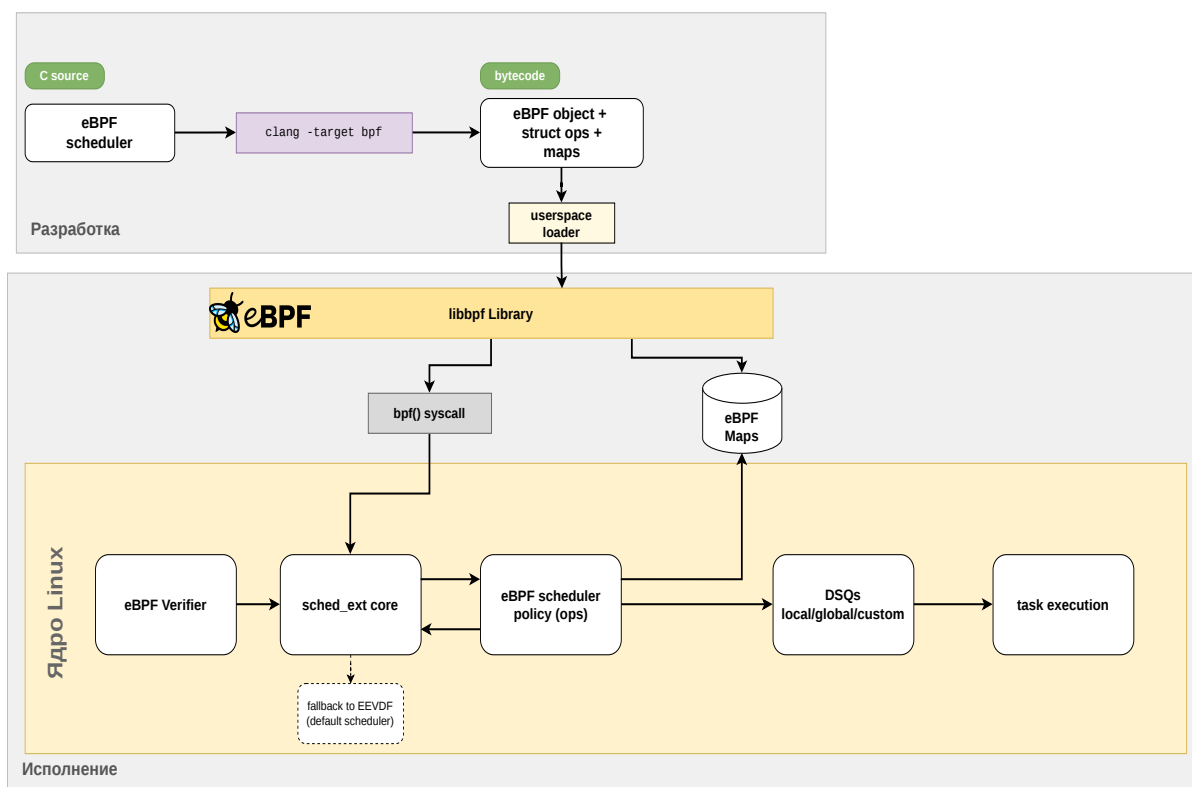


Рис. 1.1: Архитектура sched_ext в ядре Linux

Подход с вынесением политики планирования из ядра встречался и до sched_ext. В системе ghOSt [12] политика реализована как пользовательский

процесс-агент. Через разделяемую память он наблюдает состояние задач и назначает следующую задачу на процессорное ядро, а ядро ОС лишь выполняет переключение контекста. Это позволяет описывать политику на обычном языке программирования и опираться на пользовательские средства отладки.

Важно отметить, что `sched_ext` замещает только `fair`-класс. Задачи реального времени (`SCHED_FIFO`, `SCHED_RR`) и задачи с дедлайнами (`SCHED_DEADLINE`) остаются под управлением штатных классов `rt_sched_class` и `dl_sched_class`. Это ограничивает область применимости пользовательских политик, но сохраняет гарантии для критичных по времени задач.

Безопасность исполнения произвольной политики обеспечивается несколькими механизмами времени выполнения. Таймер `watchdog` отслеживает случаи, когда задача не получает процессорного времени дольше установленного порога (по умолчанию 30 секунд). При штатной выгрузке, критической ошибке внутри планировщика или срабатывании `watchdog` ядро автоматически возвращает все задачи под управление `EEVDF`, что обеспечивает безопасность экспериментов на работающей системе. Дополнительно реализована возможность экстренного отключения пользовательской политики комбинацией клавиш `SysRq-S`.

С точки зрения программной модели, политика планирования в `sched_ext` описывается набором обратных вызовов, объединённых в структуру `struct sched_ext_ops`. К основным обработчикам относятся `select_cpu`, вызываемый при пробуждении задачи и отвечающий за выбор ядра исполнения, `enqueue`, помещающий задачу в очередь готовых к выполнению, `dispatch`, выбирающий следующую задачу для конкретного ядра, а также `running` и `stopping`, дающие дополнительные контрольные точки для обновления метрик (при начале и завершении исполнения задачи). Дополнительные обработчики `init_task`, `exit_task`, `cpu_online` и `cpu_offline` позволяют поддерживать собственное состояние, связанное с задачами и ядрами, на протяжении всего их жизненного цикла. Отдельная группа обработчиков `cgroup_init`, `cgroup_exit`, `cgroup_move` и `cgroup_set_weight` обеспечивает интеграцию с иерархией контрольных групп. Они вызываются при создании и удалении `cgroup`, перемещении

задач между группами, а также при изменении веса группы, что позволяет реализовывать иерархические политики планирования.

Поведение фреймворка настраивается через флаги поля `ops->flags`. Флаг `SCX_OPS_SWITCH_PARTIAL` переводит под управление `sched_ext` только задачи с явной политикой `SCHED_EXT`, оставляя `SCHED_NORMAL`, `SCHED_BATCH` и `SCHED_IDLE` в `fair`-классе. Флаг `SCX_OPS_ENQ_LAST` определяет, попадает ли последняя выполнявшаяся задача обратно в очередь после исчерпания кванта. Флаг `SCX_OPS_KEEP_BUILTIN_IDLE` указывает ядру продолжать обновлять битовую маску простаивающих CPU при загруженной пользовательской политике. Без этого флага eBPF-программе пришлось бы вести учёт самостоятельно, тогда как с ним поиск свободного ядра доступен через штатные `kfunc`-вызовы (стабильные функции ядра, доступные из eBPF).

Передача задач между обработчиками происходит через очереди диспетчеризации *dispatch queue* (DSQ). Помимо встроенных глобальной `SCX_DSQ_GLOBAL` и локальных по числу ядер `SCX_DSQ_LOCAL`, eBPF-программа может создавать произвольное количество собственных очередей с заданным идентификатором при помощи `scx_bpf_create_dsq`. Каждая DSQ обслуживается по принципу FIFO либо по виртуальному времени, задаваемому при вставке через `scx_bpf_dsq_insert_vtime`. В режиме виртуального времени очередь реализована как красно-чёрное дерево, упорядоченное по полю `vtime`, что обеспечивает извлечение задачи с наименьшим виртуальным временем за $O(\log n)$. Возможность создавать произвольные DSQ позволяет завести отдельную очередь на каждый кластер ядер и обслуживать её по собственному виртуальному времени. Тогда задача, помещённая в очередь нужного кластера на этапе `enqueue`, попадёт на исполнение только к ядрам этого кластера, а упорядочивание по `vtime` сохраняется внутри кластера независимо от остальных. Такое разделение существенно в контексте гетерогенных архитектур, где P/E-ядра отличаются по производительности и требуют отдельного учёта.

Для взаимодействия с ядром eBPF-программа использует `kfunc`-вызовы. Так, `scx_bpf_dsq_insert` помещает задачу в выбранную очередь, а `scx_bpf_dsq_move_to_local` извлекает задачу для исполнения на текущем ядре. Состояние задач и ядер хранится в `BPF_MAP`-структурах, а доступ к полям `task_struct` и `rq` осуществляется через механизм CO-RE (Compile

Once – Run Everywhere), что обеспечивает переносимость скомпилированной программы между версиями ядра.

Для типичных нужд балансировки фреймворк предоставляет встроенный учёт простаивающих ядер. Функции `scx_bpf_pick_idle_cpu` и `scx_bpf_test_and_clear_cpu_idle` позволяют без собственной реализации находить свободные CPU по битовой маске. Если обработчик `select_cpu` помещает задачу непосредственно в локальную очередь `SCX_DSQ_LOCAL` целевого ядра, последующий вызов `enqueuee` для этой задачи пропускается, что образует быстрый путь пробуждения и сокращает накладные расходы на часто пробуждающихся задачах.

Собственное состояние, привязанное к конкретной задаче, удобно хранить в структуре типа `BPF_MAP_TYPE_TASK_STORAGE`, которая обеспечивает константное время доступа без хеш-поиска и автоматически освобождается при завершении задачи. Длительность кванта процессорного времени задаётся присваиванием поля `p->scx.slice` в обработчиках `enqueuee` или `dispatch`. По умолчанию используется `SCX_SLICE_DFL` (20 мс). Вытеснение уже исполняющейся задачи на удалённом ядре инициируется вызовом `scx_bpf_kick_cpu` с флагом `SCX_KICK_PREEMPT`.

На этапе загрузки программы среда eBPF накладывает на политику планирования ряд архитектурных ограничений, проверяемых верификатором. В программе запрещены блокирующие вызовы и динамическая аллокация памяти, любой цикл должен иметь верифицируемую верхнюю границу, а доступ к структурам ядра разрешён только через утверждённый набор `kfunc` и полей. Эти требования заметно сказываются на выборе структур данных и способов агрегирования метрик при разработке планировщика.

1.4. Актуальные исследования планировщиков

С появлением фреймворка `sched_ext` [10] исследовательская активность в области планировщиков задач заметно возросла. Большинство современных работ используют `sched_ext` как экспериментальную платформу для быстрой проверки новых алгоритмов без модификации ядра. Диапазон применения охватывает оптимизации существующих алгоритмов, подходы машинного обучения и нестандартные постановки задач.

Tang et al. [13] предложили sched_ext-планировщик SRAND, в котором расчёт отставаний и виртуальных дедлайнов EEVDF заменён упрощённой схемой виртуального времени и многоуровневой очередью. По завершении кванта виртуальное время задачи увеличивается на $(slice - p.slice) \cdot 100/p.weight$, где $slice = 20$ мс — длительность кванта, $p.slice$ — его неиспользованный остаток, $p.weight$ — вес задачи. У задач с большим весом виртуальное время растёт медленнее, что соответствует более высокому приоритету. Новые и пробуждающиеся задачи инициализируются текущим глобальным виртуальным временем системы, которое подтягивается до максимума активных задач. Если у пробудившейся задачи виртуальное время меньше $V(t) - slice$, оно поднимается до этого порога, что не даёт долго неактивной задаче монополизировать ядро.

Полученное виртуальное время определяет распределение по пяти BPF-очередям с порогами 20, 50, 200 и 500 мс (queue0–queue4). Задачи из очередей с меньшим индексом первыми переходят в единую глобальную FIFO-очередь, откуда ядра напрямую извлекают следующую. Это заменяет обход красно-чёрного дерева EEVDF одной операцией взятия с головы. Дополнительно поддерживаются локальные очереди свободных ядер с привязкой задач к ядрам, на которых они уже работали. Поток ядра получает неограниченный квант и направляется непосредственно в локальную очередь. Политика реализована через обработчики select_cpu, enqueue и dispatch. Время ожидания в очереди ограничено 5000 мс. По данным авторов, на Lmbench время переключения контекста сокращается на 11,83%, а на hackbench общая производительность растёт на 7,02% при сопоставимой с EEVDF равномерности распределения нагрузки (дисперсия загрузки ядер $s^2 = 0,005$ против 0,006 у EEVDF).

Xu et al. [14] применили sched_ext к задаче управляемого тестирования параллелизма (*controlled concurrency testing*, CCT) ядра Linux. Планировщик LACE переключает конфликтующие потоки в контролируемом порядке, автоматически вставляя точки переключения в операциях с разделяемой памятью и примитивах синхронизации, что позволяет систематически воспроизводить состояния гонки без вмешательства гипервизора. По сравнению с фаззером SegFuzz LACE увеличивает покрытие ветвлений на 38%, снижает накладные расходы на 57% и ускоряет воспроизведение ошибок в 11,4 ра-

за, обнаружив восемь ранее неизвестных ошибок параллелизма при объёме патчей ядра менее 25 строк. Работа показывает, что `sched_ext` пригоден не только для оптимизации производительности, но и как инструмент верификации корректности параллельного кода.

Мао et al. [15] рассмотрели задачу планирования с точки зрения обеспечения полноты данных о происхождении (*provenance*). При высоком темпе генерации событий EEVDF, LAVD и Rusty теряют от 39% до свыше 98% событий, что компрометирует гарантии эталонного монитора и создаёт угрозу суперпродюсера. Планировщик Aegis строится на двухслойной архитектуре. Первый слой делит задачи на основную и несколько непервичных очередей. Для непервичных задаётся время ожидания, играющее роль бюджета, а выбор очереди происходит по наибольшему времени ожидания, что обеспечивает справедливость без явных приоритетов. Второй слой управляет переключением через DeepQ-Network с двойным вознаграждением. Награда r_c штрафует за потерю событий и обеспечивает полноту, r_p поощряет утилизацию процессора. На бенчмарках с системами Sysdig и eAudit Aegis показал нулевую потерю событий при суммарных накладных расходах менее 0,5% в типичных режимах и около 2,44% в наиболее тяжёлых, что достигается дельта-функцией пропуска несущественных решений планирования.

Wang et al. [16] развили идею адаптивного планирования до уровня портфеля политик. Предложенный агент Adaptive Scheduling Agent (ASA) декомпозирует задачу на распознавание шаблона нагрузки и сопоставление ему планировщика из набора предобученных экспертов. Шаблон определяется ансамблевым классификатором на основе XGBoost по метрикам утилизации процессора, ввода-вывода и сетевой активности, собираемым через eBPF и `procfs`, а для подавления шумов применяется взвешенное по времени вероятностное голосование с экспоненциальным затуханием. Подготовка агента ведётся в три этапа: прототипное обучение базовой модели, динамическая калибровка стоимости переключения и обучение модели обобщения. Динамическое переключение `sched_ext`-политики выполняется на лету. ASA превосходит EEVDF в 86,4% тестовых сценариев и попадает в тройку лучших политик в 78,9% случаев, а на смешанных нагрузках обгоняет даже статический оракул за счёт переключения политик внутри одной задачи. Агент потребляет около 1,45% одного ядра и 297 МБ памяти и сохраняет качество при

переносе на новые аппаратные платформы, не входившие в обучающую выборку. Работа подчёркивает ограниченность статичной политики при росте разнообразия нагрузок и оборудования.

Galin и Hedblom [17] оценили планировщик LAVD (Latency-criticality Aware Virtual Deadline) в telco-окружении Kubernetes Minikube. Поводом к работе послужила проблема задержек в облачной инфраструктуре Ericsson, резко возрастающих при загрузке процессора выше 50%, что делает оценку поведения планировщика на интервале 50–95% критически важной. LAVD изначально проектировался для игровых нагрузок и предлагается для telco в силу схожих требований к задержке. Для воспроизводимой оценки авторы реализовали многопоточный симулятор на C++, генерирующий нагрузки на основе трассировочных данных Ericsson. Распределение времени обслуживания подобрано как бета-распределение, давшее наибольшее значение $p = 0,084$ по критерию Колмогорова–Смирнова, и дополнено отдельным генератором фоновой интерферирующей нагрузки.

В исследовании сопоставлены три планировщика: LAVD, минималистичный `scx_simple` без поддержки вытеснения и штатный EEVDF в роли базовой линии. При высокой доле фоновой интерферирующей нагрузки LAVD сокращает среднее время оборота относительно EEVDF на 6–81% и улучшает 99-й перцентиль до 90%. Однако в типичных для Ericsson условиях (50% загрузки трафиком и 10% интерферирующей нагрузки) `scx_simple` стабильно обгоняет LAVD, несмотря на меньшую сложность, а попытки тонкой настройки минимального и максимального кванта LAVD не дают значимых улучшений. Авторы делают вывод, что преимущество принадлежит не отдельной политике, а самой платформе `sched_ext` как средству быстрого сравнения алгоритмов и адаптации планирования к доменной специфике.

Перечисленные работы охватывают оптимизацию очередей, тестирование параллелизма, обучаемые политики и доменную адаптацию, но исходят из однородного вычислительного ресурса. Задача планирования на гетерогенных процессорах остаётся значительно менее изученной. Ближе всего к ней стоит работа Van Craeynest et al. [18], в которой интенсивность обращений к памяти признана недостаточным критерием сопоставления задачи типу ядра, а оценка производительности выводится из совместного учёта параллелизма на уровне инструкций (ILP, instruction-level parallelism) и парал-

лелизма на уровне обращений к памяти (MLP, memory-level parallelism) на уровне CPI-компонент (cycles per instruction). Предложенный механизм PIE (Performance Impact Estimation) прогнозирует поведение задачи на альтернативном ядре по CPI-стеку без её миграции и даёт прирост пропускной способности на 5,5% над современными планировщиками и на 8,7% над политиками на основе сэмплирования. Однако работа относится к 2012 году и предлагает аппаратный механизм оценки, тогда как в настоящей работе тот же учёт ведётся программно. Учёт вычислительной мощности ядер в программной политике планирования на современных гетерогенных платформах заполняет этот пробел и составляет предмет настоящей работы.

1.5. Анализ подходов к реализации планировщика

Способ реализации планировщика определяет темп исследования, цену ошибки в политике и переносимость результатов между версиями ядра. К настоящему моменту в практике закрепились четыре подхода: прямая модификация штатного fair-класса, загружаемый модуль ядра, пользовательский агент по схеме ghOSt и eBPF-программа в рамках sched_ext.

Прямая модификация штатного fair-класса обеспечивает наилучшее быстроедействие, поскольку политика выполняется без промежуточных слоёв и имеет полный доступ к внутренним структурам ядра. Однако цикл разработки оказывается дорогим. Каждое изменение требует пересборки ядра (порядка 10 минут на современной машине) и перезагрузки целевой системы. Ошибка в логике планирования может привести к панике ядра, зависанию или повреждению состояния системы. Сравнение нескольких вариантов превращается в линейное накопление времени сборки и перезагрузки. Подход оправдан при подготовке итоговой реализации к включению в основную ветку Linux, но плохо подходит для итеративного исследования.

Загружаемый модуль ядра сокращает цикл итерации до секунд, поскольку вместо пересборки ядра достаточно перезагрузки модуля. Однако последствия ошибок остаются прежними. Модуль исполняется с привилегиями ядра, и некорректная политика способна привести к тем же последствиям, что и встроенная реализация. Кроме того, в Linux отсутствует официальный интерфейс для замены планировщика fair-класса, поэтому мо-

дуль вынужден опираться на внутренние структуры ядра, такие как `struct sched_class` и поля `task_struct`, регулярно меняющиеся между версиями. ABI (Application Binary Interface) ядра в общем случае не стабилизирован, и решение оказывается жёстко привязано к конкретной версии ядра и требует сопровождения при каждом обновлении.

Пользовательский агент по схеме ghOSt [12] переносит логику планирования в обычный процесс, что обеспечивает устойчивость к ошибкам политики и широкий выбор языков реализации. Взаимодействие с ядром построено на разделяемой памяти и транзакционной передаче решений, поэтому накладные расходы ниже наивной модели на системных вызовах. Однако каждое решение требует синхронизации между ядром и агентом, и подход проигрывает реализации внутри ядра на нагрузках с короткими и частыми пробуждениями. Помимо этого, ghOSt поставляется отдельным набором патчей ядра, не входящим в основную ветку, поэтому требует сопровождения при каждом обновлении ядра. Google, основной разработчик ghOSt, прекратил его поддержку и перешёл на `sched_ext` в Android [11].

eBPF-программа в рамках `sched_ext` снимает основные ограничения предыдущих подходов. Программа выполняется внутри ядра, обеспечивая сопоставимое со встроенной реализацией быстродействие. Безопасность гарантируется верификатором eBPF на этапе загрузки (см. раздел 1.3), а таймер watchdog при зависании политики автоматически выгружает eBPF-планировщик и возвращает систему под управление EEVDF. Подключение и выгрузка планировщика выполняются на работающей системе за миллисекунды без перезагрузки, а механизм CO-RE обеспечивает переносимость скомпилированной программы между близкими версиями ядра. Кроме того, с момента принятия в основную ветку `sched_ext` стал фактическим стандартом сравнения новых алгоритмов планирования, что упрощает воспроизведение и прямое сопоставление результатов с существующими работами.

Результаты сравнения сведены в таблицу 1.1, где знаки +, — и ± обозначают полное, отсутствующее и частичное выполнение критерия. Устойчивость к ошибкам политики здесь означает отсутствие риска паники, зависания или повреждения состояния ядра при некорректной логике планирования, а поддерживаемый интерфейс замены — наличие официального ABI

для подключения внешнего планировщика без правки внутренних структур ядра.

Таблица 1.1: Сравнение подходов к реализации планировщиков

Критерий	В ядре	Модуль	ghOSt	sched_ext
Без пересборки ядра	—	+	±	+
Устойчивость к ошибкам политики	—	—	+	+
Низкие накладные расходы	+	+	—	±
Поддерживаемый интерфейс замены	—	—	±	+
Переносимость между версиями ядра	—	—	±	+

Среди рассмотренных подходов только sched_ext одновременно обеспечивает работу без пересборки ядра, устойчивость к ошибкам политики, поддерживаемый интерфейс замены и переносимость между версиями ядра, проигрывая встроенной реализации лишь по накладным расходам. Сочетание этих свойств делает возможным итеративное исследование на работающей системе и прямое сопоставление результатов с другими работами, поэтому в качестве основы для дальнейшей реализации выбран фреймворк sched_ext.

Глава 2. Математические модели планировщиков

2.1. Введение

EEVDF [5] — модель пропорционально-долевого планирования, лежащая в основе штатного планировщика Linux. Она оперирует понятиями виртуального времени, отставания и виртуального дедлайна, обеспечивая доказуемо оптимальные границы справедливости при однородном вычислительном ресурсе. В настоящей работе используется как базовая для сравнения.

Аукционная модель A1349 расширяет постановку задачи планирования на случай гетерогенного ресурса. Она опирается на динамический VCG-механизм распределения ресурсов [19] и трактует выбор типа ядра как задачу аукционного распределения недолговечного блага между конкурирующими агентами. Гетерогенность ядер встроена в саму функцию ценности и в правило аллокации, а бюджеты ограничивают суммарный платёж задачи.

2.2. Обозначения и допущения

В таблицах 2.1–2.3 приведена сводка обозначений, используемых в настоящей и последующих главах.

Таблица 2.1: Обозначения модели A1349. Платформа

Символ	Описание
$\mathcal{P}, \mathcal{E}$	Множества Р-ядер и Е-ядер
K_P, K_E	Число Р-ядер и Е-ядер
K	Общее число ядер, $K = K_P + K_E$
K_P^t, K_E^t	Число свободных ядер каждого типа в момент t
κ	Тип ядра, $\kappa \in \{P, E\}$
η_P, η_E	Производительность Р-ядра и Е-ядра
σ	Отношение производительностей, $\sigma = \eta_P/\eta_E > 1$
c_P, c_E	Базовая стоимость одного кванта на Р-ядре и Е-ядре
γ	Отношение стоимостей, $\gamma = c_P/c_E > 1$

Таблица 2.2: Обозначения модели A1349. Задачи и механизм

Символ	Описание
t	Момент времени (дискретный)
$\mathcal{N}^t$	Множество активных задач в момент t
$\theta_i = (v_i, l_i)$	Тип (характеристики) задачи i
v_i	Ценность завершения задачи i для пользователя
$\bar{V}$	Верхняя граница ценности задачи
l_i	Требуемое число квантов задачи i на Р-ядре, $l_i \in \{1, \dots, L\}$
L	Верхняя граница длины задачи на Р-ядре
$m_\kappa(l)$	Длина контракта задачи длины l на ядре типа $\kappa \in \{P, E\}$: $m_P(l) = l$, $m_E(l) = \lceil l\sigma \rceil$
B_i	Начальный бюджет задачи i
B_i^t	Остаточный бюджет в момент t : $B_i^t = B_i - \sum_{s < t} p_i^s$
χ_i	Класс обслуживания задачи i (real-time, interactive, batch, background)
s^t	Состояние MDP: $(\mathbf{x}^t, \mathbf{b}^t)$
$\mathbf{x}^t$	Остаточные длительности контрактов на ядрах
$\mathbf{b}^t$	Остаточные бюджеты активных задач
$a_i^t = (a_{i,P}^t, a_{i,E}^t)$	Назначение задачи i на тип ядра (компонент действия a^t)
$\phi_P(\theta_i), \phi_E(\theta_i)$	Эффективная ценность задачи на Р/Е-ядре, $\phi_\kappa = v_i - c_\kappa m_\kappa(l_i)$
$W(s^t)$	Социальная функция благосостояния (значение Беллмана)
$W_{-i}(s^t)$	Благосостояние без участия задачи i
δ	Коэффициент дисконтирования, $\delta \in (0, 1)$
p_i^t	VCG-платёж задачи i в момент t
π	Политика планирования
$u_i(\pi)$	Индивидуальная полезность задачи i при политике π (Допущение 9)
$\bar{W}_\kappa$	Ожидаемое будущее благосостояние для ядра типа κ

Таблица 2.3: Обозначения модели EEVDF

Символ	Описание
t	Момент времени (непрерывный)
w_i	Вес (приоритет) клиента i
$\mathcal{A}(t)$	Множество активных клиентов (задач) в момент t
q	Верхняя граница длины запроса (квант процессорного времени)
$f_i(t)$	Идеальная мгновенная доля ресурса для клиента i
$S_i(t_0, t_1)$	Идеальное обслуживание клиента i на интервале $[t_0, t_1]$
$s_i(t_0, t_1)$	Фактически полученное обслуживание клиента i
t_0^i	Момент входа клиента i в систему
$\text{lag}_i(t)$	Отставание клиента i : $S_i - s_i$
$V(t)$	Виртуальное время системы
$r^{(k)}$	Длина запроса k (в единицах обслуживания)
$ve^{(k)}$	Виртуальное время доступности запроса k
$vd^{(k)}$	Виртуальный дедлайн запроса k
$u^{(k)}$	Фактически использованная часть запроса k
$r_{\max}^k$	Максимальная длина запроса клиента k

Далее перечислены допущения, общие для обеих моделей.

1. Все ядра одного типа считаются идентичными по производительности, различие существует только между Р/Е кластерами.
2. Зависимости между задачами по данным и синхронизации не учитываются моделью.
3. Планирование выполняется в онлайн-режиме, решение о назначении задачи принимается в момент её поступления без информации о будущих задачах.
4. Производительность ядер постоянна, η_P и η_E не изменяются во времени.

5. В каждый квант ядро исполняет ровно одну задачу, задача не может занимать несколько ядер одновременно.
6. Квант неделим, вытеснение происходит только на границе квантов.
7. Задача может исполняться на любом ядре (Р или Е), миграция между типами допускается только между квантами.
8. Контекстное переключение считается мгновенным, его стоимость не моделируется.
9. Контракт длиной $m_\kappa(l_i)$ квантов в модели атомарен. После аллокации не вытесняется и не прерывается до завершения. Полезность v_i начисляется только при полном исполнении контракта. Класс задач с непрерывным приростом полезности внутри контракта (мягкое реальное время) выходит за рамки модели.
10. Задачи не стратегические агенты. Стратегическими агентами выступают владельцы задач, сообщающие v_i . Длина l_i оценивается планировщиком по истории исполнения и считается наблюдаемой, не приватной. Класс χ_i верифицируется при создании задачи и в стратегическом канале не участвует.

2.3. Earliest Eligible Virtual Deadline First

Earliest Eligible Virtual Deadline First (EEVDF) [5] — модель пропорционально- долевого планирования, предложенная Ion Stoica и Hussein Abdel-Wahab.

Рассматривается один разделяемый ресурс (процессор), время на котором выделяется квантами длительности не более q . Множество клиентов (задач), активных в момент t , обозначается $\mathcal{A}(t)$, каждому клиенту i назначен положительный вес w_i . Идеальная мгновенная доля ресурса для клиента i определяется как

$$f_i(t) = \frac{w_i}{\sum_{j \in \mathcal{A}(t)} w_j}.$$

В идеализированной непрерывной модели при $q \rightarrow 0$ обслуживание

$S_i(t_0, t_1)$, положенное клиенту i на интервале $[t_0, t_1]$, есть интеграл от f_i :

$$S_i(t_0, t_1) = \int_{t_0}^{t_1} f_i(\tau) d\tau.$$

Разность между положенным и фактически полученным обслуживанием задаёт центральную метрику справедливости, называемую отставанием (lag):

$$\text{lag}_i(t) = S_i(t_0^i, t) - s_i(t_0^i, t),$$

где t_0^i — момент входа клиента i в систему, s_i — фактически полученное обслуживание. Положительный lag соответствует недообслуженному клиенту, отрицательный — переобслуженному.

Виртуальное время системы $V(t)$ вводится как

$$V(t) = \int_0^t \frac{d\tau}{\sum_{j \in \mathcal{A}(\tau)} w_j}. \quad (2.1)$$

Пока клиент i активен на $[t_1, t_2]$, обслуживание линейно по виртуальному времени:

$$S_i(t_1, t_2) = w_i \cdot (V(t_2) - V(t_1)),$$

поскольку $\int_{t_1}^{t_2} w_i / W(\tau) d\tau = w_i (V(t_2) - V(t_1))$, где $W(\tau) = \sum_{j \in \mathcal{A}(\tau)} w_j$. За одну единицу виртуального времени каждый активный клиент получает ровно w_i единиц реального обслуживания.

Каждый запрос k клиента i описывается тремя величинами: длиной $r^{(k)}$, виртуальным временем доступности $ve^{(k)}$ и виртуальным дедлайном $vd^{(k)}$. Для краткости индекс (k) опускается, когда речь идёт об одном запросе. Запрос считается *доступным* (eligible) в момент t , если $ve \leq V(t)$.

Время доступности определяет момент, начиная с которого запрос может быть обслужен. Оно выбирается так, чтобы переобслуженный клиент ожидал, пока его идеальное обслуживание сравняется с фактически полученным:

$$ve = V(t_0^i) + \frac{s_i(t_0^i, t)}{w_i}. \quad (2.2)$$

При входе клиента i в момент t_0^i первый запрос получает $ve^{(1)} = V(t_0^i)$, что

соответствует нулевому отставанию в момент входа.

Виртуальный дедлайн выбирается так, чтобы за интервал $[ve, vd]$ в непрерывной модели клиент получал ровно r единиц обслуживания:

$$vd = ve + \frac{r}{w_i}. \quad (2.3)$$

После исполнения запроса k с использованной частью $u^{(k)} \leq r^{(k)}$ доступность следующего запроса обновляется как $ve^{(k+1)} = ve^{(k)} + u^{(k)}/w_i$. В частном случае полного исполнения ($u^{(k)} = r^{(k)}$) получаем $ve^{(k+1)} = vd^{(k)}$.

Правило выбора очередной задачи состоит из двух стадий. Сначала применяется фильтр доступности $ve \leq V(t)$, затем среди доступных запросов выбирается тот, у которого виртуальный дедлайн vd минимален. Алгоритм допускает реализацию с помощью дополненного двоичного дерева поиска со сложностью $O(\log n)$ на каждую операцию (выбор, вставку, удаление).

Лемма 1 (доступность недообслуженного клиента) [5]. *Если $\text{lag}_k(t) \geq 0$ для активного клиента k , то k имеет доступный запрос в момент t .*

Из $\text{lag}_k(t) \geq 0$ следует $s_k(t_0^k, t) \leq S_k(t_0^k, t)$. Тождество $S_k(t_0^k, t) = w_k(V(t) - V(t_0^k))$ сохраняется и при динамических событиях за счёт коррекций V , рассматриваемых ниже, откуда $ve = V(t_0^k) + s_k/w_k \leq V(t_0^k) + S_k/w_k = V(t)$.

Лемма 2 [5]. *В любой момент t сумма отставаний всех активных клиентов равна нулю:*

$$\sum_{i \in \mathcal{A}(t)} \text{lag}_i(t) = 0.$$

Следствие 1 [5]. Из Лемм 1 и 2 при непустом $\mathcal{A}(t)$ существует хотя бы один доступный запрос, поэтому EEVDF является work-conserving алгоритмом, то есть ресурс не простаивает при наличии активных клиентов.

При динамических событиях (вход, выход клиентов, изменение весов) виртуальное время системы корректируется для сохранения инварианта Леммы 2. При выходе клиента j в момент t

$$V(t^+) = V(t) + \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t^+)} w_i}, \quad (2.4)$$

при повторном входе клиента, сохранившего своё отставание $\text{lag}_j(t)$ с предыдущей сессии,

$$V(t^+) = V(t) - \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t^+)} w_i}. \quad (2.5)$$

При изменении веса клиента j с w_j на w'_j операция эквивалентна последовательности выход–вход с новым весом:

$$V(t^+) = V(t) + \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t)} w_i - w_j} - \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t)} w_i - w_j + w'_j}. \quad (2.6)$$

В [5] рассматриваются три стратегии обработки динамических событий, различающиеся тем, как поступать с отставанием клиента при входе и выходе. Первая сохраняет отставание и корректирует V по приведённым формулам, обеспечивая наилучшую долгосрочную справедливость. Вторая обнуляет отставание при выходе. Третья допускает события только при нулевом отставании, что гарантирует непрерывность V и упрощает анализ. Границы отставания, приведённые ниже, доказаны для третьей стратегии.

Для формулировки границ отставания вводится понятие стационарной системы [5].

Определение. Система называется стационарной, если все динамические события (вход, выход, изменение веса) происходят при нулевом отставании участника.

В стационарной системе виртуальное время $V(t)$ непрерывно.

Лемма 3 (граница дедлайна) [5]. В стационарной системе любой запрос активного клиента k с дедлайном d обслуживается не позднее $d + q$, где q — размер кванта.

Теорема 1 (границы отставания) [5]. В стационарной системе для любого активного клиента k отставание ограничено как

$$-r_{\max} < \text{lag}_k(t) < \max(r_{\max}, q),$$

где $r_{\max}$ — максимальная длина запроса клиента k . Обе границы асимптотически точны.

Асимметрия объясняется природой EEVDF. Переобслуживание ($\text{lag} < 0$) ограничено снизу величиной $-r_{\max}$: после полного исполнения запроса

виртуальное время доступности продвигается на r/w_k , и клиент теряет право на обслуживание до истечения собственного интервала. Недообслуживание ($\text{lag} > 0$) ограничено сверху величиной $\max(r_{\max}, q)$: ожидание возникает лишь пока обслуживается чужой запрос (не дольше q) либо пока виртуальное время не догонит ve собственного длинного запроса.

Следствие 2 (равномерный квант) [5]. *Если все запросы клиента k не превышают кванта, $r^{(k)} \leq q$, то*

$$-q < \text{lag}_k(t) < q.$$

Лемма 4 (оптимальность границ) [5]. *Для любого пропорционально- долевого алгоритма с квантами размера q существуют конфигурации, в которых $\text{lag}_k(t)$ сколь угодно близок к $-q$, и конфигурации, в которых он сколь угодно близок к q .*

Доказательство строится на примере n клиентов с равными весами: клиент, получивший первый квант, имеет $\text{lag} = q/n - q \rightarrow -q$ при $n \rightarrow \infty$, а клиент, получивший последний из первых n квантов, имеет $\text{lag} = q - q/n \rightarrow q$. Следовательно, границы $(-q, q)$ Следствия 2 неулучшаемы в классе пропорционально-долевых алгоритмов с квантом q , и EEVDF их достигает.

В стационарной системе с постоянным составом клиентов EEVDF эквивалентен алгоритму Earliest Deadline First (EDF). Фильтр доступности ($ve \leq V(t)$) отличает EEVDF от алгоритмов справедливой очередизации (Packet-by-Packet Fair Queueing), ограничивая отставание до $O(1)$ вместо $O(n)$ [5].

2.4. Аукционная модель A1349

В отличие от EEVDF, рассматривающего вычислительный ресурс как однородный, аукционная модель A1349 трактует выбор ядра как задачу распределения недолговечного ресурса между конкурирующими агентами. Формализация опирается на модель динамического механизма Ryuji Sano [19] для ресурсов с различной длительностью использования.

Платформа и агенты.

Гетерогенная платформа описывается двумя множествами ядер,

$$\mathcal{P} = \{P_1, \dots, P_{K_P}\}, \quad \mathcal{E} = \{E_1, \dots, E_{K_E}\},$$

где

- $\mathcal{P}$ — ядра высокой производительности (P-cores),
- $\mathcal{E}$ — энергоэффективные ядра (E-cores).

Время разделено на дискретные кванты $t = 1, 2, \dots$. В каждый момент времени ядро представляет собой *недолговечный* (perishable) ресурс (неиспользованный квант теряется безвозвратно).

Каждая задача $i \in \mathcal{N}^t$ характеризуется типом

$$\theta_i = (v_i, l_i),$$

где

- $v_i \in [0, \bar{V}]$ — ценность завершения задачи для пользователя,
- $l_i \in \{1, \dots, L\}$ — требуемое число квантов на P-ядре.

Расстановка агентности и канал сообщения механизма зафиксированы в Допущении 10: термины «агент» и «аукцион» относятся к владельцу задачи, а не к задаче как объекту планирования. Поскольку P/E-ядра разделяют общий набор инструкций, задача может быть назначена на любой класс ядер (выбор типа является решением механизма, а не задачи). На E-ядре длина задачи увеличивается в $\sigma > 1$ раз и составляет $\lceil l_i \cdot \sigma \rceil$ квантов, где $\sigma = \eta_P / \eta_E$ выражает отношение производительностей. Задача i получает полезность v_i только при исполнении полного контракта длиной $m_\kappa(l_i)$ квантов.

Каждой задаче назначен начальный бюджет $B_i > 0$, определяемый её классом обслуживания (real-time, interactive, batch, background). Остаточный бюджет в момент t обозначается

$$B_i^t = B_i - \sum_{s < t} p_i^s.$$

Аллокация задаче i в момент t допустима, если VCG-платёж не превышает остаточного бюджета, $p_i^t \leq B_i^t$. После победы на аукционе платёж амортизируется по квантам начатого контракта (см. ниже), что обеспечивает $B_i^{t+m} \geq 0$ к моменту его завершения. Принадлежность к классу χ_i является внешним параметром и не входит в тип θ_i .

MDP-формулировка.

Процесс планирования формализуется как марковский процесс принятия решений (MDP). Состояние в момент t записывается в виде

$$s^t = (\mathbf{x}^t, \mathbf{b}^t),$$

где

- $\mathbf{x}^t \in \{0, \dots, \lceil L\sigma \rceil - 1\}^K$ — остаточные длительности контрактов на каждом из $K = K_P + K_E$ ядер (верхняя граница учитывает максимальную длину контракта на E-ядре),
- $\mathbf{b}^t$ — остаточные бюджеты активных задач.

Действие a^t задаёт для каждой задачи распределение по типам ядер,

$$a_i^t = (a_{i,P}^t, a_{i,E}^t) \in \{0, 1\}^2, \quad a_{i,P}^t + a_{i,E}^t \leq 1,$$

с ограничениями на число свободных ядер каждого типа K_P^t, K_E^t ,

$$\sum_{i \in \mathcal{N}^t} a_{i,P}^t \leq K_P^t, \quad \sum_{i \in \mathcal{N}^t} a_{i,E}^t \leq K_E^t.$$

Следующее состояние определяется функцией перехода $s^{t+1} = G(s^t, a^t, \mathcal{N}^{t+1})$, где $\mathcal{N}^{t+1}$ — множество вновь прибывших задач. Длительность контракта на каждом ядре обновляется по правилу

$$x_k^{t+1} = \begin{cases} m_\kappa(l_i) - 1, & \text{если ядро } k \text{ назначено задаче } i, \\ \max(x_k^t - 1, 0), & \text{иначе.} \end{cases}$$

Остаточные бюджеты уменьшаются на величину платежа,

$$b_i^{t+1} = b_i^t - p_i^t.$$

Контракты и стоимость.

Базовая стоимость одного кванта на ядре типа $\kappa \in \{P, E\}$ обозначается c_P, c_E , причём $c_P > c_E$ (отношение $\gamma = c_P/c_E$ введено в таблице обозначений). Полная стоимость контракта для задачи i :

$$\text{cost}(i, P) = c_P \cdot l_i, \quad \text{cost}(i, E) = c_E \cdot \lceil l_i \cdot \sigma \rceil.$$

Из-за округления вверх отношение $\lceil l_i \sigma \rceil / l_i$ зависит от l_i и не равно константе σ . Е-ядро оказывается дешевле Р-ядра, если экономия на стоимости кванта перевешивает удлинение контракта,

$$\frac{\lceil l_i \sigma \rceil}{l_i} < \gamma,$$

и дороже в обратном случае. Поэтому выбор класса ядра зависит от длины задачи l_i и не сводится к сравнению σ с γ .

Эффективная ценность задачи на каждом типе ядра с учётом стоимости квантов:

$$\phi_P(\theta_i) = v_i - c_P \cdot l_i, \quad \phi_E(\theta_i) = v_i - c_E \cdot \lceil l_i \cdot \sigma \rceil. \quad (2.7)$$

Расширенная социальная функция благосостояния удовлетворяет уравнению Беллмана:

$$W(s^t) = \max_{a^t} \left[\sum_{i \in \mathcal{N}^t} (a_{i,P}^t \phi_P(\theta_i) + a_{i,E}^t \phi_E(\theta_i)) + \delta \cdot \mathbb{E}[W(s^{t+1})] \right],$$

где $\delta \in (0, 1)$, коэффициент дисконтирования. В настоящей работе используется дисконтированная формулировка, следуя Ryujі Sano [19]. Платёж в виде (2.8), содержащий $\delta^{m_\kappa(l)}$, имеет смысл только при $\delta < 1$.

Динамический VCG-механизм.

Обозначим $m_\kappa(l)$ длину контракта задачи на ядре типа κ : $m_P(l) = l$, $m_E(l) = \lceil l \cdot \sigma \rceil$.

Платёж задачи i , получившей ядро типа κ в момент t , определяется по правилу VCG (Викри–Кларка–Гровса) как разность между социальным бла-

гом без её участия и вкладом в текущее распределение:

$$p_i^t = W_{-i}(s^t) - W(s^t) + \phi_\kappa(\theta_i).$$

Здесь $W(s^t)$ — суммарное социальное благосостояние, включающее реализованный вклад $\phi_\kappa(\theta_i)$ победителя i , а $W_{-i}(s^t)$ обозначает оптимальное значение функции Беллмана той же MDP, в которой оптимизация проводится по всем задачам, кроме i . Формула эквивалентна стандартной маргинальной разности $W_{-i}(s^t) - [W(s^t) - \phi_\kappa(\theta_i)]$: благосостояние без участия i минус благосостояние с исключённым вкладом i . Интуитивно задача оплачивает экстерналию, налагаемую ею на остальные активные задачи. В Беллмановой формулировке полная стоимость контракта учитывается однократно в момент аллокации. Для согласования с бюджетным ограничением $\sum_s p_i^s \leq B_i$ единый платёж p_i^t разносится равными долями $p_i^s = p_i^t / m_\kappa(l_i)$ по квантам контракта. Это разнесение является номинальным и сохраняет $\sum_s p_i^s = p_i^t$, но не дисконтированную сумму $\sum_s \delta^{s-t} p_i^s$. Строгая эквивалентность с правилом индифферентности контрактов Sano [19] (два контракта с одинаковой дисконтированной суммой $\sum_s \delta^{s-t} p_i^s$ неразличимы для агента) требует дисконтированно взвешенного распределения долей. В настоящей работе используется равномерное разнесение как приближение, оправданное малыми $1 - \delta$ на временных масштабах одного контракта.

В частном случае одного свободного ядра типа κ и двух претендующих задач i^* (победитель) и j (вторая по эффективной ценности) расчёт VCG-платежа принимает форму, аналогичную однородной модели Sano [19]. Без агента i^* ядро было бы выделено j на $m_\kappa(l_j)$ квантов, после чего ожидаемое будущее благосостояние составит $\delta^{m_\kappa(l_j)} \bar{W}_\kappa$, где $\bar{W}_\kappa$ — ожидаемое значение Беллмана при возвращении ядра типа κ в свободное состояние. В присутствии i^* ядро занимает на $m_\kappa(l_{i^*})$ квантов, что даёт $\delta^{m_\kappa(l_{i^*})} \bar{W}_\kappa$. Замена оценки V_j из однородной модели на эффективную стоимость $\phi_\kappa(\theta_j) = v_j - c_\kappa m_\kappa(l_j)$ (линейная стоимость ресурса уже вычтена в социальной функции благосостояния), получаем

$$p_{i^*} = \phi_\kappa(\theta_j) + (\delta^{m_\kappa(l_j)} - \delta^{m_\kappa(l_{i^*})}) \bar{W}_\kappa. \quad (2.8)$$

Формула адаптирует результат [19] на гетерогенную платформу с отдельным учётом будущего благосостояния по типу ядра.

Свойства механизма.

Свойства совместимости стимулов и индивидуальной рациональности рассматриваются ниже только по координате v_i , поскольку длина l_i выступает наблюдаемым параметром окружения и соответствующие условия Sano [19] по этой координате сводятся к тождеству.

Sano [19] (Теорема 1) приводит необходимые и достаточные условия байесовской совместимости стимулов динамического прямого механизма с типом (v_i, l_i) через монотонность вероятности аллокации и интегральную структуру выплат, а Предложение 1 той же работы усиливает результат до доминирующих стратегий для VCG-механизма с детерминированным монотонным распределением. В принятой расстановке агентности эти условия применяются по координате v_i и задают монотонность VCG-аллокации, что образует операционный антистимул к завышению веса агентом-владельцем. Завышенный v_i повышает шанс победы на аукционе и одновременно увеличивает VCG-платёж, списываемый из бюджета владельца, что делает правдивое сообщение предпочтительной стратегией. Класс обслуживания χ_i задаётся вне канала сообщений механизма и в анализе совместимости стимулов не участвует.

Индивидуальная рациональность в смысле Sano [19] (Теорема 2) для VCG-механизма с оптимальной политикой приводит к выражению полезности агента как маргинального вклада в социальное благосостояние и наследуется стандартным образом.

Предложение 1 (бюджетная допустимость).

Если VCG-платёж превышает остаточный бюджет, $p_i^(s, a) > B_i^t$, задача не получает ядро в данном кванте. Модифицированное распределение $\tilde{a}^t = \arg \max_a \sum_{i: p_i \leq B_i^t} [a_{i,P} \phi_P(\theta_i) + a_{i,E} \phi_E(\theta_i)] + \delta \mathbb{E}[W(s^{t+1})]$ максимизирует социальное благосостояние на множестве бюджетно-допустимых аллокаций.*

Жёсткое бюджетное ограничение в общем случае несовместимо с правилом VCG-платежей. Исключение задачи при $p_i^* > B_i^t$ нарушает монотонность распределения по v_i и выводит механизм за рамки условий байесовской совместимости стимулов Sano [19] (Dobzinski, Lavi, Nisan [20] о бюд-

жетных аукционах). При расстановке агентности из Допущения 10 Предложение 1 формулируется как свойство аллокации, а не как утверждение о совместимости стимулов. Бюджет ограничивает суммарную экстерналию, которую агент-владелец может наложить на систему, а исчерпание бюджета переводит задачу в категорию без претензий на сохранение совместимости стимулов (IC).

Таким образом, модель A1349 предлагает альтернативу пропорционально-долевому планированию — аукционный механизм, учитывающий гетерогенность ядер, бюджетные ограничения задач и динамику системы во времени.

Глава 3. Реализация алгоритмов планирования

3.1. Реализация EEVDF с помощью sched_ext

Данный раздел описывает реализацию математической модели EEVDF (раздел 2.3) в виде планировщика для платформы sched_ext. Представленная здесь реализация имеет вспомогательный характер и приводится для демонстрации общего подхода к отображению математических моделей планировщиков на платформу sched_ext.

Переменные состояния. Глобальное состояние системы хранится в BPF_MAP-структуре типа BPF_MAP_TYPE_ARRAY с единственным элементом структуры eevdf_ctx. Структура содержит два поля:

- vtime_now — системное виртуальное время $V(t)$ (уравнение (2.1));
- total_weight — суммарный вес активных задач $\sum_{j \in \mathcal{A}(t)} w_j$ (знаменатель $V(t)$).

Виртуальное время хранится в формате с фиксированной точкой и масштабируется коэффициентом SCALE = 1000. Среда eBPF не поддерживает арифметику с плавающей точкой, а приращения $V(t)$ при умеренной нагрузке имеют величину порядка единицы и обнулились бы при прямом целочисленном делении на $\sum w_j$. Масштабирование на SCALE сохраняет три младших разряда дроби и устраняет потерю точности. Операции сравнения $ve \leq V(t)$ при этом остаются корректными, так как обе величины хранятся в одинаковых единицах.

Состояние отдельной задачи хранится в BPF_MAP-структуре типа BPF_MAP_TYPE_TASK_STORAGE, что обеспечивает обращение за $O(1)$ без хеш-поиска и автоматическое освобождение при завершении задачи. Структура task_ctx содержит виртуальное время доступности ve (соответствует $ve^{(k)}$ в уравнении (2.2)), виртуальный дедлайн vd ($vd^{(k)}$, уравнение (2.3)) и флаг нахождения в очереди on_rq. Поля ve и vd , как и глобальное vtime_now, хранятся в одних и тех же масштабированных единицах виртуального времени.

Единственная очередь диспетчеризации SHARED_DSQ создаётся при инициализации и упорядочена по виртуальному дедлайну vd . При вставке через `scx_bpf_dsq_insert_vtime` ядро использует поле `vtime` как ключ красно-чёрного дерева, что реализует правило выбора EEVDF — среди доступных запросов извлекается тот, чей vd минимален.

Поверх SHARED_DSQ фреймворк `sched_ext` предоставляет на каждое ядро встроенную локальную очередь `SCX_DSQ_LOCAL`, из которой планировщик ядра берёт следующую задачу для исполнения. Перенос задач из общей очереди в локальные выполняется в обработчике `dispatch` вызовом `scx_bpf_dsq_move_to_local`: при освобождении ядра вызывается `dispatch` для этого ядра, который переносит голову SHARED_DSQ (задачу с минимальным vd) в его локальную очередь. На быстром пути пробуждения `select_cpu` задача может быть вставлена напрямую в `SCX_DSQ_LOCAL` свободного ядра, минуя SHARED_DSQ; в этом случае обработчик `enqueue` для данной задачи пропускается.

Отображение обработчиков на модель. В таблице 3.1 приведено соответствие между обратными вызовами `sched_ext` и шагами алгоритма EEVDF.

Ключевые аспекты реализации.

Атомарное продвижение $V(t)$. Системное виртуальное время обновляется при каждом снятии задачи с процессора (обработчик `stopping`). Поскольку на многоядерной системе несколько ядер могут одновременно вызывать `stopping`, продвижение V выполняется через атомарную операцию `__sync_fetch_and_add`. Величина приращения

$$\Delta V = \frac{u \cdot \text{SCALE}}{\sum_j w_j},$$

где u — фактически использованная часть кванта (`SCX_SLICE_DFL - p->scx.slice`), является дискретной формой непрерывного правила $dV/dt = 1/\sum w_j$ из уравнения (2.1).

Двусторонний ограничитель отставания. Реализация фиксирует длину запроса равной размеру кванта, $r = q = \text{SCX_SLICE_DFL}$ (20 мс), что согласует дискретное продвижение времени с границами Теоремы 1. При по-

Таблица 3.1: Соответствие обработчиков sched_ext модели EEVDF

Обработчик	Реализуемый шаг модели
select_cpu	Быстрый путь пробуждения при $ve \leq V(t)$ на свободном ядре
enqueue	Ограничитель отставания (теорема 1) и вставка по $vd = ve + q/w_i$ (уравнение (2.3))
dispatch	Извлечение задачи с минимальным vd
stopping	Продвижение ve , vd и $V(t)$ на $\Delta V = u / \sum w_j$ (уравнение (2.1))
enable	Вход задачи: $ve := V(t)$, $\sum w_j += w_i$ (уравнение (2.5))
disable	Выход задачи: коррекция $V(t)$ по уравнению (2.4)
set_weight	Коррекция $V(t)$ по уравнению (2.6)
running	Пустой обработчик
init_task	Выделение task_ctx
init	Создание SHARED_DSQ, инициализация eevdf_ctx
exit	Передача кода завершения агенту (UEI, User Exit Info)

становке задачи в очередь (обработчик enqueue) применяется ограничение

$$V(t) - q_v \leq ve' \leq V(t) + q_v,$$

где $q_v = q \cdot \text{SCALE}/w_i$ — виртуальная стоимость одного кванта. В коде ограничение реализовано условиями $ve' < V(t) - q_v \Rightarrow ve' := V(t) - q_v$ и $ve' > V(t) + q_v \Rightarrow ve' := V(t) + q_v$, что эквивалентно приведённой формуле. Нижняя граница не позволяет задаче, накопившей значительный положительный lag за время сна, монополизировать ресурс по возвращении. Верхняя граница не позволяет задаче уйти в далёкое будущее виртуального времени.

Теорема 1 даёт асимметричные границы $-r_{\max} < \text{lag}_k(t) < \max(r_{\max}, q)$, тогда как реализованный ограничитель симметричен. При принятом условии $r = q$ обе границы совпадают по модулю с q , поэтому симметричное ограничение $|V - ve| \leq q_v$ накрывает их с запасом и не нарушает асимптотических оценок. Симметричная форма выбрана ради простоты eBPF-кода и постоянного числа ветвлений в обработчике.

Обработка динамических событий. Реализация следует первой из трёх стратегий, описанных в разделе 2.3: отставание задачи сохраняется при выходе и переносится в момент возврата, а виртуальное время системы корректируется по формулам Леммы 2 для сохранения инварианта $\sum \text{lag}_i = 0$. При входе задачи (enable) её виртуальное время доступности инициализируется как $ve = V(t)$, что соответствует нулевому отставанию в момент входа. Суммарный вес обновляется атомарно. При выходе (disable) виртуальное время корректируется по уравнению (2.4): $V(t^+) = V(t) + \text{lag} / \sum w_{j \neq i}$. В реализации lag вычисляется в виртуальных единицах как $V(t) - ve$ и связан с теоретическим определением $\text{lag}_i = S_i - s_i$ множителем $1/w_i$. Знак при этом сохраняется: положительное значение означает недообслуженность как в модели, так и в реализации.

При изменении веса (set_weight) применяется двухшаговая коррекция по уравнению (2.6), эквивалентная последовательности выход–вход с новым весом.

Таким образом, реализация сохраняет все инварианты модели EEVDF при ограничениях среды eBPF: отсутствии блокирующих вызовов, конечных циклах и фиксированных структурах данных. Единственное отклонение от теоретической модели состоит в замене непрерывного продвижения $V(t)$ дискретными приращениями на границах квантов, что не нарушает границ теоремы 1, поскольку q конечно.

3.2. Реализация аукционной модели A1349

Данный раздел описывает реализацию аукционной модели A1349 (раздел 2.4) в виде eBPF-планировщика scx_A1349, построенного средствами sched_ext. Задачи во всех очередях упорядочены непосредственно по эффективной ценности $\phi_\kappa(\theta_i)$, а одноюнитный VCG-платёж по формуле (2.8) рассчитывается на этапе диспетчеризации путём чтения двух старших кандидатов из кластерной DSQ через итератор bpf_iter_scx_dsq.

Атомарный контракт против вытесняемой реализации. Допущение 9 требует, чтобы контракт длиной $m_\kappa(l_i)$ квантов исполнялся целиком и не прерывался до завершения. В реализации это требование не выполняется: контракт служит лишь горизонтом планирования аукциона, а не атомарной

единицей исполнения. Вытеснение допускается между квантами, маршрутизация $\kappa^* = \arg \max_{\kappa} \phi_{\kappa}$ пересматривается при пробуждении задачи, а на границе кванта длительные задачи могут быть перенаправлены в привязанные к ядрам очереди (см. ниже). Расхождение компенсируется EWMA-оценкой $\bar{W}_{\kappa}$, накапливаемой по реализованным значениям ϕ , и стационарностью выбора кластера в пределах одного пробуждения.

Операционная редукция модели. Полный аппарат раздела 2.4 предполагает, что задачи сообщают тип $\theta_i = (v_i, l_i)$ через канал механизма, а оптимальное распределение строится по точному значению функции Беллмана $W(s^t)$ с известным переходным ядром MDP. В реализации эти предпосылки ослабляются: ценность v_i наблюдается ядром непосредственно через штатные интерфейсы операционной системы — `nice`, `sched_setattr` и `cgroup.cpu.weight` (отображаемые на `p->scx.weight`), длина l_i оценивается по EWMA-среднему наблюдаемой истории исполнения, а переходное ядро остаётся неизвестным. VCG-аппарат сохраняет двойную роль, задавая правило максимизации общественного благосостояния $\kappa^* = \arg \max_{\kappa} \phi_{\kappa}$ и формулу платежа (2.8) как операционный антистимул к завышению v_i агентом-владельцем за счёт списания из бюджета. Точное встречное благосостояние $W_{-i}(s^t)$ заменяется кластерной EWMA-оценкой $\bar{W}_{\kappa}$ реализованных ϕ , обновляемой в обработчике `stopping`. Все расчёты выполняются за $O(1)$ на событие и укладываются в ограничения верификатора eBPF (раздел 1.3) благодаря предвычисленной в пользовательском пространстве таблице δ^m .

Перечисленные упрощения — замена W_{-i} на экспоненциально взвешенную оценку $\bar{W}_{\kappa}$, использование веса w_i как косвенной меры ценности v_i , оценка длины l_i через экспоненциально взвешенное среднее наблюдаемой истории исполнения, бюджет на уровне задачи, а не агента-владельца — выводят реализацию за рамки строгих условий байесовской совместимости стимулов и совместимости стимулов в доминантных стратегиях Sano [19] (Теорема 1, Предложение 1). В частности, $\bar{W}_{\kappa}$ как среднее реализованных $|\phi|$ не является аппроксимацией оптимума функции Беллмана $W_{-i}(s^t)$, поэтому платёж (2.8), вычисленный по такой оценке, не сохраняет маргинального смысла VCG. Сохраняется лишь операционный антистимул: завышение v_i повышает шанс победы на аукционе и одновременно увеличивает списание из бюджета задачи, что в рамках принятого Допущения 10 о нестратегично-

сти задач достаточно для задуманной цели — предотвращения монополизации Р-кластера задачами с искусственно завышенным весом.

Архитектура очередей диспетчеризации. При инициализации создаются три кластерные DSQ и набор привязанных к ядрам очередей:

- AUCTION_DSQ_P (id = 1) — очередь задач, назначенных на Р-кластер (множество $\mathcal{P}$)
- AUCTION_DSQ_E (id = 2) — очередь задач, назначенных на Е-кластер (множество $\mathcal{E}$)
- AUCTION_DSQ_STARVED (id = 3) — очередь задач с исчерпанным бюджетом (нарушение бюджетного ограничения B_i^t)
- AUCTION_DSQ_PERCPU_BASE + CPU_NUM (id = 100 + *cru*) — по одной очереди на ядро для долго работающих задач, упорядоченная по ϕ . На этапе init создаётся AUCTION_NCPU_MAX = 64 таких очередей. Они сохраняют кэш-локальность между квантами и образуют отдельный уровень иерархии диспетчеризации в отличие от SCX_DSQ_LOCAL, который служит безусловным FIFO-слотом ближайшего исполнения.

Все четыре типа DSQ упорядочены по приоритетному ключу $\text{encode}(\phi_{\kappa^*}(\theta_i))$, который кодирует знаковую эффективную ценность в положительное число u64 как

$$\text{encode}(\phi) = \begin{cases} \text{PHI_BIAS} - \phi, & \phi \geq 0, \\ \text{PHI_BIAS} + |\phi|, & \phi < 0, \end{cases} \quad \text{PHI_BIAS} = 2^{62}.$$

Большее ϕ соответствует меньшему ключу и попадает в начало очереди, что согласуется со встроенным в sched_ext правилом сортировки DSQ по возрастанию vtime. Типичный диапазон $|\phi|$ ограничен весом задачи (после описанной ниже перенормировки $|\phi| \lesssim 5 \cdot 10^4$), что заведомо укладывается в 2^{62} .

Переменные состояния. Состояние планировщика разделено на три части, различающиеся по владельцу:

- конфигурационная BPF_MAP-структура global_data (тип BPF_MAP_

TYPE_ARRAY, значение — auction_ctx) обновляется пользовательским агентом и используется BPF только для чтения

- исполнительная BPF_MAP-структура runtime_data (тип BPF_MAP_TYPE_ARRAY, значение — auction_runtime) хранит EWMA-оценки $\bar{W}_P$, $\bar{W}_E$ и обновляется только BPF
- по-задачное состояние auction_task_ctx в BPF_MAP-структуре типа BPF_MAP_TYPE_TASK_STORAGE с флагом BPF_F_NO_PREALLOC.

Разделение конфигурационной и исполнительной структур исключает гонки между периодическим обновлением параметров платформы пользовательским пространством и накоплением EWMA-оценок в BPF.

В таблице 3.2 приведены основные переменные состояния и их соответствие символам модели.

Таблица 3.2: Переменные состояния планировщика A1349

Переменная	Символ модели	Описание
max_capacity, min_capacity	η_P, η_E	Вычислительная мощность P- и E-ядер из cru_capacity
cost_p, cost_e	c_P, c_E	Стоимости одного кванта на ядре каждого типа
p_core_count, e_core_count	K_P, K_E	Число P/E ядер на платформе
w_bar_p, w_bar_e	$\bar{W}_P, \bar{W}_E$	EWMA-оценка ожидаемого будущего благосостояния на каждом кластере
phi_enq	$\phi_{\kappa^*}(\theta_i)$	Эффективная ценность задачи, выбранная при постановке в очередь
m_enq	$m_{\kappa}(l_i)$	Длина контракта, используемая в индексе таблицы δ^m
budget	B_i^t	Остаточный бюджет задачи

Переменная	Символ модели	Описание
budget_max	$B_i^{\max}$	Максимальный бюджет ($w_i \cdot \text{BUDGET_MUL}$)
len_est_ns	l_i (нс-прокси)	EWMA длительности исполнения, прокси для l_i
on_p_type	$\kappa(i_t)$	Тип кластера, на который задача назначена в текущем кванте
long_slice	—	Признак выдачи Е-кванта (для учёта в stopping)
last_stop_ns	—	Момент последнего истинного засыпания, основа для пополнения бюджета
wake_prev_cpu	—	Подсказка кэш-привязки, сохранённая в select_cpu
delta_table[m]	δ^m	Предвычисленная в пользовательском пространстве таблица степеней дисконта (fixed-point, шкала 2^{20})
cpu_is_p[cpu]	—	Классификация ядер на Р/Е, заполняется пользовательским агентом

Отображение обработчиков на модель. Соответствие обработчиков шагам аукционного механизма приведено в таблице 3.3.

Таблица 3.3: Соответствие обработчиков sched_ext аукционной модели

Обработчик	Реализуемый шаг модели
set_weight	Пересчёт $B_i^{\max}$ и пропорциональное масштабирование остаточного бюджета

Обработчик	Реализуемый шаг модели
select_cpu	P-bias сканирование простаивающих ядер (см. ниже). При найденном свободном ядре прямая вставка в SCX_DSQ_LOCAL с P-квантом (аукцион пропускается ввиду отсутствия конкуренции). Запоминается prev_cpu для кэш-привязки на медленном пути
enqueue	Вычисление ϕ_P, ϕ_E (уравнение (2.7)); выбор кластера κ^* при пробуждении (иначе сохранение ранее выбранного); длина контракта $m_\kappa(l_i)$; бюджетная проверка B_i^t (при $B_i^t < B_i^{\max}/10$ задача направляется в AUCTION_DSQ_STARVED); асимметричное P→E перенаправление коротких задач; привязанная к ядру очередь для длительных вытесненных задач; кэш-привязка к prev_cpu при пробуждении
dispatch	Четырёхуровневое извлечение: привязанная к ядру очередь, далее аукцион в локальном кластере, далее кросс-кластерное заимствование, далее STARVED (см. ниже)
running	Пустой обработчик
stopping	Учёт фактически потраченного времени кванта; обновление EWMA длительности len_est_ns ($\alpha = 1/8$); обновление EWMA $\bar{W}_\kappa$ ($\alpha = 1/16$); фиксация last_stop_ns только при истинном засыпании (runnable = false)
enable	Приём задачи в планировщик: $B_i^{\max} = w_i \cdot \text{BUDGET_MUL}$, $B_i^0 = B_i^{\max}$ (полностью оплачена при входе), len_est_ns = SLICE_P
disable	Удаление состояния задачи через bpf_task_storage_delete
init	Создание кластерных DSQ, очереди STARVED, набора AUCTION_NCPU_MAX = 64 привязанных к ядрам очередей; заполнение значений стоимостей по умолчанию

Вычисление эффективной ценности. Модель определяет эффективную ценность задачи на ядре типа κ как (уравнение (2.7)):

$$\phi_P(\theta_i) = v_i - c_P \cdot l_i, \quad \phi_E(\theta_i) = v_i - c_E \cdot \lceil l_i \cdot \sigma \rceil.$$

В реализации v_i аппроксимируется весом задачи w_i (`p->scx.weight`), а длина l_i приводится к безразмерным Р-квантам через нормировку наносекундной EWMA длительности:

$$l_q = \frac{\hat{l}_{ns}}{\text{SLICE_P}}.$$

Получаемые компоненты ϕ имеют тот же масштаб, что и веса задач:

$$\phi_P = w_i - c_P \cdot l_q, \quad \phi_E = w_i \cdot \frac{\eta_E}{\eta_P} - c_E \cdot l_q.$$

Нормировка делением на SLICE_P продиктована совместимостью шкал. Прямая формула $\phi \sim w \cdot l_{ns}$ давала бы $|\phi| \sim 10^{10}$ при одновременном масштабе бюджета $\sim w \cdot \text{BUDGET_MUL} \sim 10^6$, что приводило бы VCG-платежи либо к отсечению в нуле, либо к одноразовому опустошению корзины токенов в одном такте диспетчеризации. После перенормировки $|\phi| \lesssim 5 \cdot 10^4$ совпадает по порядку с диапазоном весов `p->scx.weight` $\in [1, 49152]$ и согласуется как со шкалой бюджета, так и со шкалой платежей. Дисконт ценности на Е-ядре множителем η_E/η_P отражает то, что Е-квант доставляет в σ^{-1} раз меньше полезной работы (эквивалентно теоретическому штрафу $c_E \cdot \lceil l_i \sigma \rceil$ с противоположной стороны равенства). Деление наносекундной стоимости на длину кванта SLICE_P выполняется с округлением (прибавлением SLICE_P/2 до целочисленного деления), что исключает систематическое смещение для подквантовых задач.

На уровне диспетчеризации реализация использует две длительности гранения кванта: `AUCTION_SLICE_P` = 20 мс для Р-кластера и `AUCTION_SLICE_E` = $(3 \cdot \text{SLICE_P})/2$ = 30 мс для Е-кластера. Увеличенный квант на Е амортизирует постоянные накладные расходы переключения контекста, относительный вклад которых выше на медленном ядре, и снижает

частоту перепланирования длительных вычислительных задач.

Длина контракта вычисляется как $l_p = \lceil \text{len_ns} / \text{SLICE_P} \rceil$, $l_p \geq 1$, с $m_P(l) = l_p$ и $m_E(l) = \lceil l_p \cdot \eta_P / \eta_E \rceil$ (округление вверх). Значение насыщается на $\text{MAX_CONTRACT_LENGTH} - 1 = 31$, что соответствует горизонту $\approx 0,64$ с при Р-кванте 20 мс ($32 \cdot 20$ мс) и согласуется с предположением модели о конечной верхней границе L .

Бюджетный механизм. Каждой задаче назначается бюджет в виде корзины токенов (token bucket), моделирующей ограничение B_i из раздела 2.4. Максимальный бюджет пропорционален весу: $B_i^{\max} = w_i \cdot \text{BUDGET_MUL}$, где $\text{BUDGET_MUL} = 2000$. При входе задачи в планировщик (обработчик enable) бюджет инициализируется полным значением, $B_i^0 = B_i^{\max}$ — задача принимается в систему «полностью оплаченной». Пополнение выполняется при пробуждении (флаг SCX_ENQ_WAKEUP) пропорционально длительности сна:

$$\Delta B_i = \min(\Delta t_{\text{idle}}, T_{\max}) \cdot \frac{w_i}{\text{REPLENISH_DIV}},$$

где $T_{\max} = 1$ с ($\text{REPLENISH_IDLE_CAP}$) и $\text{REPLENISH_DIV} = 5 \cdot 10^6$. Бюджет отсекается сверху значением $B_i^{\max}$. Если на этапе enqueue остаточный бюджет $B_i^t < B_i^{\max} / 10$ (в коде эквивалентно $10 \cdot B_i^t < B_i^{\max}$), задача направляется напрямую в $\text{AUCTION_DSQ_STARVED}$, не тратя такт диспетчеризации на заведомо неоплатоспособный аукцион. Это реализует Предложение 2.4 в виде дешёвой консервативной проверки на входе. Окончательная проверка $p_{i^*} \leq B_{i^*}^t$ выполняется на этапе диспетчеризации с учётом фактического платежа.

Точный VCG-платёж. Платёж рассчитывается по формуле (2.8) непосредственно, без приближения резервной ценой. При вызове обработчика dispatch из кластерной DSQ через `bpf_iter_scx_dsq` извлекаются две головные задачи: победитель i^* и претендент j (вторая по ϕ задача). Платёж в реализации представлен как

$$p_{i^*} = \phi_{\kappa}(\theta_j) + \frac{\delta^{m_{\kappa}(l_j)} - \delta^{m_{\kappa}(l_{i^*})}}{\text{DELTA_SCALE}} \cdot \bar{W}_{\kappa},$$

где значения δ^m извлекаются из таблицы `delta_table` (fixed-point со шкалой $\text{DELTA_SCALE} = 2^{20}$), предвычисленной в пользовательском пространстве на основе настраиваемого параметра δ (по умолчанию $\delta = 0,98$, $L = 32$, го-

ризонт $\sim 0,64$ с при Р-кванте 20 мс). Если очередь содержит единственного претендента, полагается $\phi_j = 0$, $m_j = 0$ (то есть $\delta^0 = 1$), что даёт вырожденную форму

$$p_{i^*} = (1 - \delta^{m_\kappa(l_{i^*})}) \bar{W}_\kappa,$$

интерпретируемую как «оплата одинокого победителя» упущенной ценностью свободной траектории ядра. Отрицательный платёж отсекается в нуле: VCG-механизм никогда не пополняет бюджет.

Если $p_{i^*} \leq B_{i^*}^t$, бюджет уменьшается на p_{i^*} , и задача i^* переносится в локальную очередь ядра SCX_DSQ_LOCAL. В противном случае задача i^* перемещается в AUCTION_DSQ_STARVED с сохранением её ϕ -кодированного ключа (через `scx_bpf_dsq_move_set_vtime` перед `scx_bpf_dsq_move_vtime`, чтобы не нарушить дисциплину сортировки DSQ), а аукцион повторяется для нового верхнего элемента той же очереди. Число попыток в пределах одного вызова `dispatch` ограничено константой `DISPATCH_AUCTION_TRIES = 3`, что согласуется с требованием верификатора eBPF к ограниченному циклам. Если победитель не может исполняться на текущем ядре (нарушение `p->cpus_ptr`, типичный случай — `per-CPU kworker`’ы), он перенаправляется в SCX_DSQ_LOCAL_ON разрешённого простаивающего ядра (через `scx_bpf_pick_idle_cpu`, при отсутствии — `scx_bpf_pick_any_cpu`) и раунд считается несостоявшимся, чтобы цикл продолжил работу со следующего верхнего элемента.

Обучение $\bar{W}_\kappa$. Оценка $\bar{W}_\kappa$ обновляется в обработчике `stopping` как экспоненциально взвешенное скользящее среднее реализованной эффективной ценности задачи:

$$W_{\text{real}} = |\phi_\kappa(\theta_i)| \cdot \frac{t_{\text{consumed}}}{\text{SLICE_P}}, \quad \bar{W}_\kappa \leftarrow \frac{(W_BAR_EWMA_DEN - 1) \cdot \bar{W}_\kappa + W_{\text{real}}}{W_BAR_EWMA_DEN},$$

где `W_BAR_EWMA_DEN = 16`. Модуль ϕ обеспечивает $\bar{W}_\kappa \geq 0$ и интерпретируется как ожидаемое будущее благосостояние, служащее множителем для δ -разности при расчёте платежа. Нормировка на `SLICE_P` устраняет смещение между квантом Р-цикла (20 мс) и удлинённым квантом Е-цикла (30 мс): фактически потраченное время кванта приводится к единой шкале «Р-квант». Длительность исполнения l_i обновляется отдельной EWMA с $\alpha = 1/8$: $\hat{l} \leftarrow (7\hat{l} + t_{\text{consumed}})/8$, более медленной для подавления шумовых колебаний ϕ

между соседними квантами.

Маршрутизация и стационарность кластера. Решение аукциона $\kappa^* = \arg \max_{\kappa} \phi_{\kappa}$ пересматривается только при пробуждении задачи (флаг SCX_ENQ_WAKEUP) либо при её первом появлении в системе (`last_stop_ns` = 0). Вытеснение по истечении кванта (`preempt re-enqueue`) сохраняет ранее выбранный кластер `tctx->on_p_type`: повторная оценка ϕ между двумя соседними квантами одной задачи внесла бы шум в маршрутизацию без новой информации, а миграция в середине пробега обнуляет кэш и ухудшает пропускную способность памяти.

Кэш-привязка к `prev_cpu` при пробуждении реализуется так: если ядро, на котором задача исполнялась в последний раз, принадлежит выбранному кластеру, входит в маску `p->cpus_ptr` и в данный момент бездействует, задача вставляется напрямую в `SCX_DSQ_LOCAL_ON | prev_cpu`, минуя кластерный аукцион. Если текущий процессор не совпадает с `prev_cpu`, дополнительно выполняется `scx_bpf_kick_cpu` с флагом `SCX_KICK_IDLE`, чтобы целевое ядро немедленно вышло из простоя. Согласованность с моделью сохраняется: пустое ядро в кластере не создаёт конкуренции, и $\arg \max \phi$ принимает все доступные задачи.

Долго работающие задачи (`len_est_ns` $\geq$ `SLICE_P`), вытесненные на границе кванта (когда обработчик `enqueue` вызывается без флага `SCX_ENQ_WAKEUP` при ненулевом `last_stop_ns`), направляются не в кластерную DSQ, а в привязанную к текущему ядру очередь `AUCTION_DSQ_PERCPU_BASE + CPU_NUM`. Это предотвращает рассеяние CPU-связанных задач по ядрам кластера между квантами и сохраняет кэш-локальность, что критично для пропускной способности и латентности памяти.

Асимметричное перенаправление. Чистый $\arg \max \phi_{\kappa}$ не учитывает числа ядер в кластерах: на платформе с $K_P = 8$, $K_E = 16$ строгое предпочтение Р-кластера оставляло бы Е-ядра простаивать. Реализация добавляет перенаправление (`spill`) на основе нормализованной глубины очереди:

$$Q_P \geq K_P \quad \text{и} \quad Q_P \cdot K_E > Q_E \cdot K_P \quad \implies \quad \text{маршрутизация в Е.}$$

Перекрёстное умножение выбрано вместо деления, чтобы избежать 64-битной операции деления на быстром пути eBPF. Перенаправление активируется только для коротких задач ($\hat{l}_{ns} < \text{SLICE_P}$). Длительные

memory-bound задачи остаются в выбранном по ϕ кластере, так как их перенос на E ведёт к двукратному росту латентности и $\sim 30\%$ потере пропускной способности памяти (эмпирически измерено на PassMark MEM/MEM_LAT). При перенаправлении пересчитываются ϕ_E и $m_E(l_i)$, чтобы DSQ-ключ соответствовал реальному кластеру назначения.

Выбор ядра при пробуждении. Обработчик `select_cpu` реализует специализированное сканирование, отличающееся от вспомогательного `scx_bpf_select_cpu_dfl`. Поведение по умолчанию имеет систематический уклон к `prev_cpu` и часто оставляет однопоточные тесты на E-ядре, в то время как для задач со среднестатистическим весом выполняется $\phi_P \geq \phi_E$. Реализация выполняет следующий порядок поиска:

1. быстрый путь: `prev_cpu` выбирается, если оно является P-ядром, входит в `p->cpus_ptr` и в данный момент простаивает;
2. полный путь: `bpf_for`-цикл по диапазону $[0, 64)$ (константа `AUCTION_NCPU_MAX`) ищет любое простаивающее P-ядро в маске задачи;
3. запасной путь: при отсутствии свободных P-ядер вызывается `scx_bpf_select_cpu_dfl`, который при необходимости возвращает простаивающее E-ядро.

Если найдено свободное ядро, задача вставляется напрямую в `SCX_DSQ_LOCAL` с P-квантом и обработчик `enqueuee` для этой задачи пропускается — образуется быстрый путь пробуждения. Поле `tctx->wake_prev_cpu` заполняется до сканирования и используется обработчиком `enqueuee` для возможного кэш-привязочного `pin`'а на медленном пути, когда `select_cpu` не смог разместить задачу на свободном ядре.

Диспетчеризация и заимствование. Обработчик `dispatch` реализует четырёхуровневый приоритет извлечения:

1. привязанная к ядру очередь `AUCTION_DSQ_PERCPU_BASE + CPU_NUM` — наиболее кэш-тёплые задачи, обслуживается первой через `scx_bpf_dsq_move_to_local`;
2. локальная кластерная очередь `AUCTION_DSQ_κ` с полноценным аукционом по формуле (2.8), бюджетной проверкой и возможным перемещением неоплатоспособной задачи в `STARVED`. Если очередь содержит

ровно одну задачу (`scx_bpf_dsq_nr_queued = 1`), аукцион вырождается в копию (нет претендента j), и реализация переходит на ускоренный путь `scx_bpf_dsq_move_to_local`, экономя выделение и освобождение итератора DSQ;

3. кросс-кластерное заимствование из `AUCTION_DSQ_k` прямым перемещением в локальную очередь ядра без повторного аукциона: нахождение задачи в чужом кластере означает, что её ϕ был рассчитан под этот кластер, а $\bar{W}_k$ заимствующего ядра внесла бы шум в платёж; сохранение работы (work conservation) важнее точности VCG в этой фазе;
4. очередь `STARVED` — последний приоритет, задачи с исчерпанным бюджетом обслуживаются ядрами обоих кластеров после пополнения через простой.

Дополнительно после успешной постановки задачи в кластерную очередь обработчик `enqueue` выполняет `work-conservation-будильник`: вызовом `scx_bpf_pick_idle_cpu` ищется любое свободное ядро из маски задачи и при его обнаружении (не совпадающем с текущим) выполняется `scx_bpf_kick_cpu` с флагом `SCX_KICK_IDLE`, чтобы вновь добавленная задача не ждала истечения кванта на соседнем ядре.

Пространство пользователя. Программа `scx_A1349.c` выполняет функции агента конфигурации. При запуске она:

- считывает мощности ядер из `/sys/devices/system/cpu/cpu*/cpu_capacity`, определяет $\eta_P = \max_{cpu} cap$, $\eta_E = \min_{cpu} cap$ и вычисляет $\sigma = \eta_P / \eta_E$
- классифицирует ядра по порогу `P_CAP_PCT = 90 %`: ядро считается P-ядром, если его `cpu_capacity` $\geq 0,9 \cdot \eta_P$, иначе E-ядром
- при отсутствии явного указания `-e` автоматически калибрует $c_E = c_P \cdot \eta_E / \eta_P$, чтобы выполнялось $\gamma = c_P / c_E = \sigma$
- вычисляет таблицу степеней дисконта $\delta^m \cdot 2^{20}$ для $m \in [0, \text{MAX_CONTRACT_LENGTH})$ в двойной точности и записывает её как `fixed-point` в `BPF_MAP`-структуру `delta_table`. Это исключает арифметику с плавающей точкой на BPF-стороне, не поддерживаемую верификатором.

Значения по умолчанию: $c_P = 1024$, $\delta = 0,98$. При запуске агент проверяет $c_P > 0$, $0 < \delta < 1$ и при пользовательском задании c_E — условие $c_P > c_E > 0$ (Р-ядро должно быть строго дороже Е-ядра). В режиме работы агент один раз в 5 секунд повторяет процедуру `refresh_cpu_capacities` для обнаружения `hotplug`-событий или динамического изменения `cpu_capacity` и обновляет `global_data` при изменении любого параметра.

Таким образом, реализация строит порядок исполнения непосредственно по эффективной ценности ϕ . Маргинальный VCG-платёж рассчитывается точно из одноюнитной формулы (2.8) с заменой неизвестного W_{-i} на EWMA-оценку $\bar{W}_\kappa$ реализованных ценностей. Дополнительные механизмы — Р-bias выбор ядра при пробуждении, кэш-привязка к `prev_cpu` с пробуждением целевого ядра, привязанные к ядрам очереди для длительных задач, асимметричное перенаправление коротких задач из перегруженного Р-кластера и ускоренный путь для одноэлементной очереди — обеспечивают практическую конкурентоспособность с EEVDF без отступления от формальной аукционной модели.

Глава 4. Описание тестового окружения и экспериментального оборудования

4.1. Тестовое оборудование и программное окружение

Экспериментальная оценка планировщика проведена на настольной платформе с гетерогенным процессором Intel Core Ultra 7 265K (кодовое название Arrow Lake-S). Характеристики платформы сведены в таблицу 4.1.

Таблица 4.1: Характеристики тестовой платформы

Параметр	Значение
Процессор	Intel Core Ultra 7 265K (Arrow Lake-S)
Р-ядра	8 × Lion Cove, до 5,5 ГГц
Е-ядра	12 × Skymont, до 4,6 ГГц
Потоков на ядро	1 (без Hyper-Threading)
Всего потоков	20
Кэш L2	3 МБ × 8 Р-ядер + 4 МБ × 3 модуля Е-ядер
Кэш L3 (общий)	30 МБ
Микрокод	0x121
Оперативная память	32 ГБ DDR5, 6000 МГц
TDP / PL1 / PL2 (Thermal Design Power, Power Limit)	125 Вт / 250 Вт
C-states	включены
Turbo Boost	включён
Turbo Boost Max 3.0	включён
Регулятор частоты	powersave (intel_pstate)
Операционная система	Arch Linux
Ядро	Linux 7.0
libbpf / sched_ext	в составе Linux 7.0
hackbench	2.10 (rt-tests)
sysbench	1.0.20
schbench	commit 6300b8f
PostgreSQL	18.3

Выбор платформы обусловлен прямым соответствием между аппаратной топологией и гетерогенной постановкой задачи. Процессор содержит два кластера ядер с различной производительностью. Отсутствие SMT (Hyper-

Threading) исключает влияние разделяемых ресурсов между аппаратными потоками внутри одного ядра и упрощает интерпретацию результатов, поскольку каждая задача получает полный набор исполнительных блоков ядра без конкуренции на уровне SMT (Hyper-Threading).

В качестве операционной системы используется Arch Linux с ядром Linux версии 7.0 и включённой поддержкой `sched_ext`. Базовым планировщиком fair-класса служит EEVDF, который в экспериментах выступает контрольной линией. Предложенные планировщики `scx_EEVDF` и `scx_A1349` подключаются к `sched_ext`. Регулятор частоты оставлен в значении по умолчанию. Управление тактовой частотой выполняется аппаратно (Hardware-managed P-states, HWP), а драйвер `intel_pstate` передаёт процессору подсказки энергоэффективности (EPP).

4.2. Набор бенчмарков

Для оценки планировщиков используется набор из трёх бенчмарков, каждый из которых генерирует характерный профиль нагрузки и измеряет отдельный аспект поведения системы. Бенчмарки запускаются последовательно в порядке `hackbench`, `sysbench`, `schbench` с интервалами охлаждения между фазами, чтобы каждый бенчмарк стартовал из одинакового исходного состояния процессора (температура и тактовые частоты) и последовательность тестов не влияла на результаты. Порядок планировщиков рандомизируется между запусками, что усредняет вклад фоновых процессов между планировщиками. Близкие работы по `sched_ext` планировщикам опираются на схожий инструментарий. Например, Tang и соавторы [13] применяют `hackbench` при оценке SRAND, планировщика на основе глобальной очереди готовых задач.

hackbench. Утилита из пакета `rt-tests`. Создаёт группы процессов, обменивающихся сообщениями через каналы или сокеты. Массовое порождение короткоживущих задач и интенсивное межпроцессное взаимодействие нагружает механизмы пробуждения, миграции и балансировки.

sysbench. Многопоточный бенчмарк с модулем `oltp_read_only`. Выполняет транзакции чтения к базе данных PostgreSQL. Потоки многократно блокируются на ожидании ответа базы и просыпаются лишь на время обра-

ботки ответа, что нагружает механизм пробуждения.

schbench. Бенчмарк задержек планирования, моделирующий веб-нагрузку. Каждый запрос рабочего потока состоит из двух фаз сна (имитация сети и диска) и фазы матричных вычислений. Потоки-координаторы ставят задания в очередь и ожидают результатов. Бенчмарк проверяет три аспекта планировщика — способность загрузить все ядра, поддержание длительных квантов без вытеснения и минимизацию задержки пробуждения. Бенчмарк наиболее чувствителен к качеству решений о размещении задач на ядрах, поскольку вытеснение потока во время матричных вычислений приводит к удержанию им блокировки в неисполняемом состоянии, что блокирует прогресс ожидающих потоков на других ядрах и снижает совокупную пропускную способность.

В дополнение к метрикам бенчмарков считываются системные счётчики и показания подсистемы Intel RAPL. Полный перечень измеряемых величин с указанием единиц измерения и целевого направления оптимизации приведён в разделе 4.3.

Каждый эксперимент проводится при трёх уровнях нагрузки — **лёгкая, умеренная и стрессовая** — различающихся параметрами запуска бенчмарков (таблица 4.2). На каждом уровне исполняются $N = 6$ запусков каждого из четырёх планировщиков, всего 24 запуска на уровень и 72 запуска в эксперименте. Порядок планировщиков на каждой итерации выбирается случайной перестановкой из четырёх элементов. Длительность одного запуска определяется параметрами бенчмарка (sysbench и schbench запускаются на 30 секунд, hachbench — до завершения заданного числа сообщений), перед каждым запуском выдерживается пауза 30 секунд для сброса частот процессора, при старте каждого бенчмарка дополнительно отбрасываются первые 5 секунд измерений в качестве разогрева. По результатам запусков вычисляются выборочные средние, выборочные стандартные отклонения и 95%-доверительные интервалы по t -распределению Стьюдента с числом степеней свободы $df = N - 1 = 5$ в предположении приближённой нормальности выборочного среднего.

Набор бенчмарков отражает три качественно различных профиля нагрузки. Hachbench создаёт массовое порождение короткоживущих задач и интенсивный обмен сообщениями, проверяя механизмы пробуждения и ба-

Таблица 4.2: Параметры запуска бенчмарков по уровням нагрузки

Бенчмарк	лёгкая	умеренная	стрессовая
hackbench	-l 10000	-g 4 -l 120000	-g 10 -l 50000
sysbench (поток)	1	4	20
schbench (-m/-t)	1 / 5	2 / 20	4 / 40

лансировки. Sysbench воспроизводит серверный режим, в котором потоки регулярно блокируются на ожидании ответа базы данных. Schbench сочетает длительные вычислительные кванты с короткими фазами сна и измеряет задержку реакции на пробуждение. Подобное сочетание позволяет одновременно оценить способность планировщика поддерживать высокий темп переключений и минимизировать задержки на гетерогенном оборудовании.

4.3. Измеряемые метрики

Полный перечень измеряемых величин приведён в таблицах 4.3 и 4.4. В столбце «Интерпретация» пометка «больше лучше» обозначает метрики, у которых большее значение соответствует лучшему поведению планировщика, «меньше лучше» — метрики, у которых лучшему поведению соответствует меньшее значение, «неоднозначно» — диагностические показатели без однозначной интерпретации.

Таблица 4.3: Метрики бенчмарков

Источник	Метрика	Единица	Интерпретация
hackbench	время выполнения	с	меньше лучше
sysbench	TPS	тр./с	больше лучше
sysbench	QPS	зап./с	больше лучше
schbench	задержка пробуждения p99	мкс	меньше лучше
schbench	задержка пробуждения p99.9	мкс	меньше лучше
schbench	задержка запроса p99	мкс	меньше лучше
schbench	задержка запроса p99.9	мкс	меньше лучше
schbench	RPS	зап./с	больше лучше

Hackbench измеряет общее время выполнения теста. Sysbench реги-

стрирует число выполненных транзакций PostgreSQL в секунду (TPS) и число SQL-запросов в секунду (QPS). Schbench возвращает 99-й и 99.9-й процентиля двух распределений — задержки пробуждения рабочего потока и задержки обслуживания запроса, а также число запросов, обслуженных в секунду (RPS).

Таблица 4.4: Системные счётчики и показания подсистемы Intel RAPL

Источник	Метрика	Единица	Интерпретация
/proc/stat	загрузка процес- сора	%	неоднозначно
/proc/schedstat	переключения контекста	шт./с	неоднозначно
/proc/schedstat	кванты в секунду	шт./с	неоднозначно
Intel RAPL	потреблённая энергия	Дж	меньше лучше

Из /proc/stat считывается доля времени, проведённого процессором вне состояния простоя. Из /proc/schedstat извлекаются частота переключений контекста и число квантов планирования в секунду. Из подсистемы Intel RAPL читается счётчик потреблённой энергии домена package (весь процессорный чип) в джоулях. Для каждого планировщика фиксируется приращение счётчика за время запуска, что позволяет сравнивать суммарные энергозатраты.

Глава 5. Анализ результатов эксперимента

В настоящей главе приводятся результаты сравнительного исследования планировщика `scx_A1349` и трёх других планировщиков на трёх уровнях нагрузки: лёгкой, умеренной и стрессовой. В качестве эталона взят штатный `EEVDF`, реализованный в основной ветви ядра Linux. Дополнительно использован `scx_EEVDF`, представляющий собой реализацию алгоритма `EEVDF` поверх `sched_ext`, выполненную в рамках выпускной квалификационной работы. Его включение позволяет отделить вклад окружения `sched_ext` и BPF-вызовов от эффектов аукционной модели `A1349`. Третий планировщик, `LAVD`, выбран как один из широко используемых в индустрии, например, он используется в исследовании телекоммуникационных рабочих нагрузок Ericsson [17]. Все значения получены усреднением по шести запускам, для каждой метрики приводится среднее, стандартное отклонение и 95%-ный доверительный интервал.

5.1. Результаты при лёгкой нагрузке

В режиме лёгкой нагрузки каждый бенчмарк запускается с минимальными параметрами параллелизма из применяемых в эксперименте. Такой режим позволяет оценить качество решений планировщика в условиях слабой конкуренции за ядра, когда определяющим фактором становится выбор типа ядра для каждой задачи.

Результаты бенчмарка `hackbench` представлены в таблице 5.1.

Таблица 5.1: `hackbench`, лёгкая нагрузка (время выполнения, секунды)

Планировщик	Среднее	Ст. откл.	95%-ДИ
<code>scx_A1349</code>	1,811	0,034	[1,775 ...1,847]
<code>scx_EEVDF</code>	1,850	0,013	[1,837 ...1,864]
штатный <code>EEVDF</code>	1,896	0,011	[1,885 ...1,907]
<code>LAVD</code>	1,976	0,009	[1,966 ...1,986]

Планировщик `scx_A1349` показывает наименьшее среднее время выполнения (1,811 с), что на 4,5 % быстрее штатного `EEVDF` (1,896 с). Дове-

рительный интервал `scx_A1349` [1,775 ... 1,847] не пересекается с интервалом штатного `EEVDF` [1,885 ... 1,907] и интервалом `LAVD` [1,966 ... 1,986], поэтому преимущество `scx_A1349` над ними статистически значимо. Интервал `scx_EEVDF` [1,837 ... 1,864] пересекается с интервалом `scx_A1349`, поэтому различие между ними статистически не значимо.

Результаты `sysbench` приведены в таблице 5.2.

Таблица 5.2: `sysbench`, лёгкая нагрузка (транзакции в секунду)

Планировщик	Среднее TPS	Ст. откл.	95%-ДИ
штатный <code>EEVDF</code>	4255	36	[4217 ... 4292]
<code>LAVD</code>	4200	30	[4169 ... 4231]
<code>scx_A1349</code>	4185	14	[4170 ... 4199]
<code>scx_EEVDF</code>	3591	216	[3364 ... 3818]

На `sysbench` при лёгкой нагрузке штатный `EEVDF` показывает наивысшее число транзакций в секунду (4255). Планировщик `scx_A1349` достигает 4185 TPS, что на 1,6 % ниже штатного `EEVDF`. Доверительный интервал `scx_A1349` [4170 ... 4199] не пересекается с интервалом штатного `EEVDF` [4217 ... 4292], поэтому отставание статистически значимо, однако численно невелико. `scx_A1349` опережает `scx_EEVDF` на 16,5 %, что подтверждает преимущество явного учёта типа ядра.

Результаты `schbench` по задержке пробуждения и средней пропускной способности приведены в таблице 5.3.

Таблица 5.3: `schbench`, лёгкая нагрузка (задержка пробуждения в мкс и средняя пропускная способность в запросах/сек)

Планировщик	p99	p99.9	Ср. RPS
<code>scx_EEVDF</code>	4	38	1761
штатный <code>EEVDF</code>	153	187	590
<code>LAVD</code>	170	207	671
<code>scx_A1349</code>	158	190	586

При лёгкой нагрузке `schbench` планировщик `scx_EEVDF` достигает рекордно низкой хвостовой задержки пробуждения ($p99 = 4$ мкс) и наивысшей пропускной способности (1761 RPS), что объясняется крайне агрессивной локальной диспетчеризацией пробуждаемых задач. Планировщик `scx_A1349` показывает $p99 = 158$ мкс, что сопоставимо с штатным `EEVDF`

(153 мкс) и LAVD (170 мкс). По хвостовой задержке p99 интервал scx_A1349 [156 ... 160] не пересекается с интервалом штатного EEVDF [152 ... 155], что указывает на значимое, но количественно небольшое различие в пользу штатного EEVDF.

Средняя пропускная способность scx_A1349 (586 RPS) сопоставима со штатным EEVDF (590), уступает LAVD (671) на 14,5 % и более чем втрое уступает scx_EEVDF (1761). Различие может объясняться тем, что при лёгкой нагрузке одиночные пробуждения schbench, по-видимому, не требуют миграции между кластерами, и аукционный механизм scx_A1349, возможно, добавляет фиксированные накладные расходы без выигрыша от оптимального назначения.

Хвостовые задержки запроса schbench представлены в таблице 5.4.

Таблица 5.4: schbench, лёгкая нагрузка (задержка запроса, мкс)

Планировщик	p99	p99.9
scx_EEVDF	8528	8789
штатный EEVDF	8464	8464
scx_A1349	8464	8592
LAVD	8480	8907

По задержке обслуживания запросов при лёгкой нагрузке четыре планировщика близки: значение p99 укладывается в диапазон 8464–8528 мкс.

Энергопотребление при лёгкой нагрузке приведено в таблице 5.5.

Таблица 5.5: Суммарное энергопотребление, лёгкая нагрузка (Дж)

Планировщик	Среднее	Ст. откл.	95%-ДИ
scx_EEVDF	2574	26	[2547 ... 2600]
scx_A1349	2623	5	[2618 ... 2628]
штатный EEVDF	2846	14	[2831 ... 2860]
LAVD	3070	15	[3054 ... 3086]

Планировщик scx_A1349 расходует 2623 Дж за запуск, что на 7,8 % меньше штатного EEVDF (2846 Дж) и на 14,6 % меньше LAVD (3070 Дж). Соответствующие доверительные интервалы не пересекаются. Энергопреимущество scx_EEVDF (2574 Дж) над scx_A1349 статистически значимо, но количественно невелико. Это может согласовываться с результатами hackbench:

возможно, более короткое время выполнения задач транслируется в меньшее общее энергопотребление.

5.2. Результаты при умеренной нагрузке

Режим умеренной нагрузки занимает промежуточное положение между лёгким и стрессовым: параметры параллелизма каждого бенчмарка увеличены относительно лёгкого режима, что создаёт ощутимую конкуренцию за ядра, но не доводит систему до перегрузки.

Результаты `hackbench` при умеренной нагрузке представлены в таблице 5.6.

Таблица 5.6: `hackbench`, умеренная нагрузка (время выполнения, секунды)

Планировщик	Среднее	Ст. откл.	95%-ДИ
<code>scx_EEVDF</code>	8,710	0,029	[8,679 ...8,741]
<code>scx_A1349</code>	8,907	0,068	[8,835 ...8,978]
штатный <code>EEVDF</code>	9,729	0,023	[9,704 ...9,753]
<code>LAVD</code>	10,147	0,034	[10,111 ...10,182]

Планировщик `scx_A1349` занимает второе место после `scx_EEVDF` (8,907 с против 8,710 с) и опережает штатный `EEVDF` на 8,5 % (8,907 с против 9,729 с). Доверительные интервалы `scx_A1349` и `scx_EEVDF` не пересекаются, поэтому различие между ними значимо. Интервалы `scx_A1349` и штатного `EEVDF` также не пересекаются. Опережение `scx_A1349` над `LAVD` составляет 12,2 % и также значимо.

Результаты `sysbench` при умеренной нагрузке приведены в таблице 5.7.

Таблица 5.7: `sysbench`, умеренная нагрузка (транзакции в секунду)

Планировщик	Среднее TPS	Ст. откл.	95%-ДИ
штатный <code>EEVDF</code>	16 631	30	[16 599 ...16 662]
<code>scx_A1349</code>	16 506	36	[16 468 ...16 545]
<code>LAVD</code>	14 357	60	[14 294 ...14 420]
<code>scx_EEVDF</code>	12 530	81	[12 446 ...12 615]

На `sysbench` при умеренной нагрузке планировщик `scx_A1349` (16 506 TPS) уступает штатному `EEVDF` (16 631 TPS) всего 0,8 %. Доверительные

интервалы `scx_A1349` [16 468 ... 16 545] и штатного `EEVDF` [16 599 ... 16 662] не пересекаются, поэтому отставание формально значимо, однако его величина незначительна с практической точки зрения. Планировщик `scx_A1349` превосходит `LAVD` на 15,0 % и `scx_EEVDF` на 31,7 %, при этом соответствующие интервалы не пересекаются. Это может подтверждать, что в этом тесте учёт гетерогенности существенно компенсирует общие накладные расходы `sched_ext`.

Результаты `schbench` по задержке пробуждения и средней пропускной способности при умеренной нагрузке представлены в таблице 5.8.

Таблица 5.8: `schbench`, умеренная нагрузка (задержка пробуждения в мкс и средняя пропускная способность в запросах/сек)

Планировщик	p99	p99.9	Ср. RPS
<code>LAVD</code>	882	1405	4314
<code>scx_EEVDF</code>	2276	2767	7547
<code>scx_A1349</code>	3091	3975	8627
штатный <code>EEVDF</code>	3339	4907	5702

На `schbench` при умеренной нагрузке планировщик `scx_A1349` достигает наивысшей пропускной способности: 8627 RPS, что на 51,3 % выше штатного `EEVDF` (5702) и на 14,3 % выше `scx_EEVDF` (7547). Доверительные интервалы средних RPS у всех четырёх планировщиков не пересекаются, поэтому ранжирование статистически значимо. По хвостовой задержке пробуждения `LAVD` удерживает лидерство по p99 (882 мкс) и p99.9 (1405 мкс), однако его пропускная способность вдвое ниже, чем у `scx_A1349`.

Хвостовая задержка пробуждения p99 у `scx_A1349` (3,09 мс) значимо ниже штатного `EEVDF` (3,34 мс) и значимо выше `scx_EEVDF` (2,28 мс) и `LAVD`. По p99.9 `scx_A1349` (3,97 мс) показывает лучшее значение, чем штатный `EEVDF` (4,91 мс), но уступает `scx_EEVDF` и `LAVD`.

Задержка обслуживания запросов `schbench` при умеренной нагрузке представлена в таблице 5.9.

Таблица 5.9: schbench, умеренная нагрузка (задержка запроса, мкс)

Планировщик	p99	p99.9
scx_A1349	10 827	11 861
scx_EEVDF	12 451	13 640
штатный EEVDF	34 453	52 629
LAVD	106 155	203 093

По задержке обслуживания запросов `scx_A1349` и `scx_EEVDF` показывают близкие хвостовые значения (p99 10,83 и 12,45 мс, p99.9 11,86 и 13,64 мс соответственно). При этом штатный EEVDF имеет p99.9 = 52,63 мс, а LAVD — 203,09 мс, что свидетельствует о длительном ожидании отдельных запросов. Планировщик `scx_A1349` обеспечивает на порядки меньшие хвостовые задержки обслуживания, чем штатный EEVDF и LAVD.

Энергопотребление при умеренной нагрузке приведено в таблице 5.10.

Таблица 5.10: Суммарное энергопотребление, умеренная нагрузка (Дж)

Планировщик	Среднее	Ст. откл.	95%-ДИ
scx_A1349	8105	31	[8072 ... 8138]
LAVD	8110	61	[8046 ... 8175]
scx_EEVDF	8191	21	[8169 ... 8213]
штатный EEVDF	8201	24	[8176 ... 8226]

Планировщик `scx_A1349` показывает наименьшее суммарное энергопотребление (8105 Дж). Доверительный интервал [8072 ... 8138] пересекается с интервалом LAVD [8046 ... 8175], поэтому различие между ними не значимо. Преимущество `scx_A1349` над штатным EEVDF составляет 1,2 % и значимо (интервалы не пересекаются). В целом различия между планировщиками по энергопотреблению в данном режиме невелики.

5.3. Результаты при стрессовой нагрузке

В режиме стрессовой нагрузки используются наибольшие из применяемых в эксперименте параметров параллелизма, что создаёт максимальную конкуренцию за вычислительные ресурсы. Данный режим позволяет оценить поведение планировщика при высокой степени загрузки системы.

Результаты `hackbench` при стрессовой нагрузке представлены в таблице 5.11.

Таблица 5.11: `hackbench`, стрессовая нагрузка (время выполнения, секунды)

Планировщик	Среднее	Ст. откл.	95%-ДИ
<code>scx_A1349</code>	9,269	0,056	[9,210 ... 9,328]
<code>scx_EEVDF</code>	9,283	0,027	[9,255 ... 9,311]
штатный <code>EEVDF</code>	9,396	0,032	[9,363 ... 9,429]
<code>LAVD</code>	9,789	0,050	[9,737 ... 9,841]

При стрессовой нагрузке планировщик `scx_A1349` показывает наименьшее время выполнения `hackbench` (9,269 с), за ним следует `scx_EEVDF` (9,283 с). Доверительные интервалы `scx_A1349` [9,210 ... 9,328] и `scx_EEVDF` [9,255 ... 9,311] пересекаются, поэтому различие между двумя планировщиками статистически не значимо. Преимущество `scx_A1349` над штатным `EEVDF` составляет 1,4 % и значимо (интервалы не пересекаются), а отставание `LAVD` от `scx_A1349` составляет 5,6 % и также значимо.

Результаты `sysbench` при стрессовой нагрузке приведены в таблице 5.12.

Таблица 5.12: `sysbench`, стрессовая нагрузка (транзакции в секунду)

Планировщик	Среднее TPS	Ст. откл.	95%-ДИ
штатный <code>EEVDF</code>	68 605	353	[68 234 ... 68 975]
<code>scx_A1349</code>	63 515	587	[62 899 ... 64 131]
<code>LAVD</code>	52 429	895	[51 490 ... 53 369]
<code>scx_EEVDF</code>	41 371	993	[40 329 ... 42 414]

На `sysbench` при стрессовой нагрузке штатный `EEVDF` достигает 68 605 TPS. Планировщик `scx_A1349` показывает 63 515 TPS, что составляет 92,6 % от штатного `EEVDF`, но на 21,2 % выше `LAVD` и в 1,54 раза выше `scx_EEVDF`. Доверительные интервалы всех четырёх планировщиков попарно не пересекаются, поэтому ранжирование статистически значимо. Отставание `sched_ext`-планировщиков от штатного `EEVDF` на смешанной нагрузке `sysbench` может объясняться накладными расходами на BPF-вызовы при высоком темпе переключений контекста и отсутствием интеграции с механизмом `wake_affine`, оптимизирующим локальность кэша при пробуждении. В пределах `sched_ext` `scx_A1349` обеспечивает существенно

более высокую пропускную способность. На стрессовом режиме `scx_A1349` удерживает темп переключений контекста, сопоставимый с `scx_EEVDF`, при существенно более высокой пропускной способности, что подтверждает эффективность аукционного назначения задач между кластерами Р- и Е-ядер.

Результаты `schbench` при стрессовой нагрузке по задержкам пробуждения и средней пропускной способности представлены в таблице 5.13.

Таблица 5.13: `schbench`, стрессовая нагрузка (задержка пробуждения в мкс и средняя пропускная способность в запросах/сек)

Планировщик	p99	p99.9	Ср. RPS
LAVD	116	20 827	1691
<code>scx_EEVDF</code>	15 989	16 544	7801
штатный EEVDF	11 072	17 056	5557
<code>scx_A1349</code>	12 763	17 547	8650

При стрессовой нагрузке планировщик LAVD удерживает самую низкую хвостовую задержку пробуждения p99 (116 мкс), однако его пропускная способность (1691 RPS) более чем в пять раз ниже, чем у `scx_A1349` (8650 RPS), а хвостовая задержка пробуждения p99.9 (20,83 мс) выше, чем у трёх остальных планировщиков. Это указывает на то, что LAVD достигает низкой p99 пробуждения за счёт замедления самих запросов. У планировщика `scx_A1349` хвостовая задержка пробуждения p99 составляет 12,76 мс. Интервалы доверительных оценок p99 `scx_A1349` [12,45 ... 13,08] мс и штатного EEVDF [10,87 ... 11,28] мс не пересекаются, поэтому отставание `scx_A1349` от штатного EEVDF по p99 значимо. По p99.9 `scx_A1349` (17,55 мс) близок к штатному EEVDF (17,06 мс) и `scx_EEVDF` (16,54 мс), значимо опережая лишь LAVD.

По средней пропускной способности `scx_A1349` лидирует с большим отрывом: 8650 запросов в секунду, что на 55,7 % выше штатного EEVDF (5557), на 10,9 % выше `scx_EEVDF` (7801) и в 5,11 раза выше LAVD (1691). При перегрузке аукционный механизм направляет задачи, чувствительные к задержке, на Р-ядра, а длительные вычисления — на Е-ядра.

Задержка обслуживания запросов `schbench` при стрессовой нагрузке представлена в таблице 5.14.

Таблица 5.14: schbench, стрессовая нагрузка (задержка запроса, мкс)

Планировщик	p99	p99.9
scx_A1349	19 040	24 512
scx_EEVDF	21 134	27 208
штатный EEVDF	445 781	1 692 331
LAVD	950 613	1 808 384

По задержке обслуживания запросов `scx_A1349` показывает наименьшие значения хвостовых процентилей среди всех четырёх планировщиков: $p99 = 19,04$ мс, $p99.9 = 24,51$ мс. У штатного EEVDF при стрессовой нагрузке хвостовые задержки значительно возрастают: $p99 = 445,78$ мс и $p99.9 \approx 1,69$ с. У планировщика LAVD соответствующие задержки ещё выше: $p99 \approx 0,95$ с, $p99.9 \approx 1,81$ с. Различия с `scx_A1349` в обоих случаях исключительно велики, а доверительные интервалы не пересекаются. При перегрузке штатный EEVDF и LAVD допускают длительное ожидание отдельных запросов, тогда как у обоих `sched_ext`-планировщиков (`scx_A1349` и `scx_EEVDF`) хвостовые задержки удерживаются в пределах нескольких десятков миллисекунд. Близость значений `scx_A1349` и `scx_EEVDF` не позволяет приписать это исключительно аукционному механизму A1349 и может быть связано также с особенностями работы `sched_ext`.

5.4. Сводное сравнение по режимам нагрузки

Ниже приведены графики, на которых для каждой метрики показаны значения по всем трём режимам нагрузки и четырём планировщикам. Порядок планировщиков на столбцах одинаков для всех графиков.

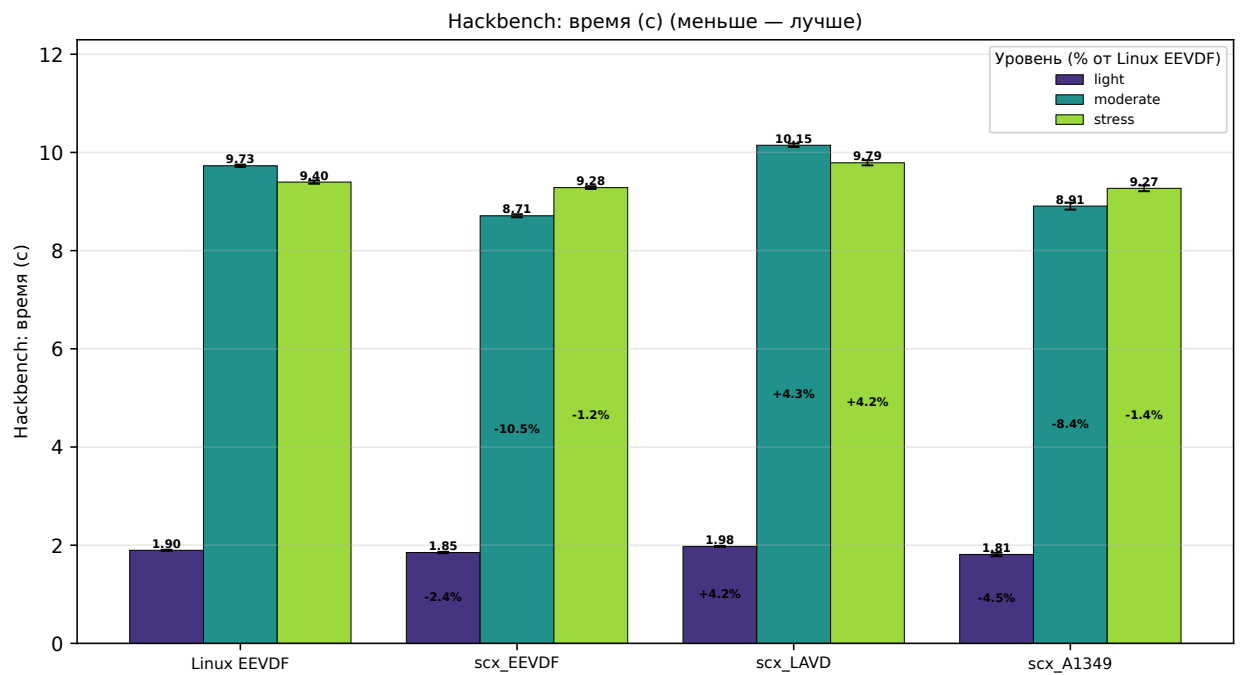


Рис. 5.1: Время выполнения hackbench (секунды) для четырёх планировщиков по трём режимам нагрузки

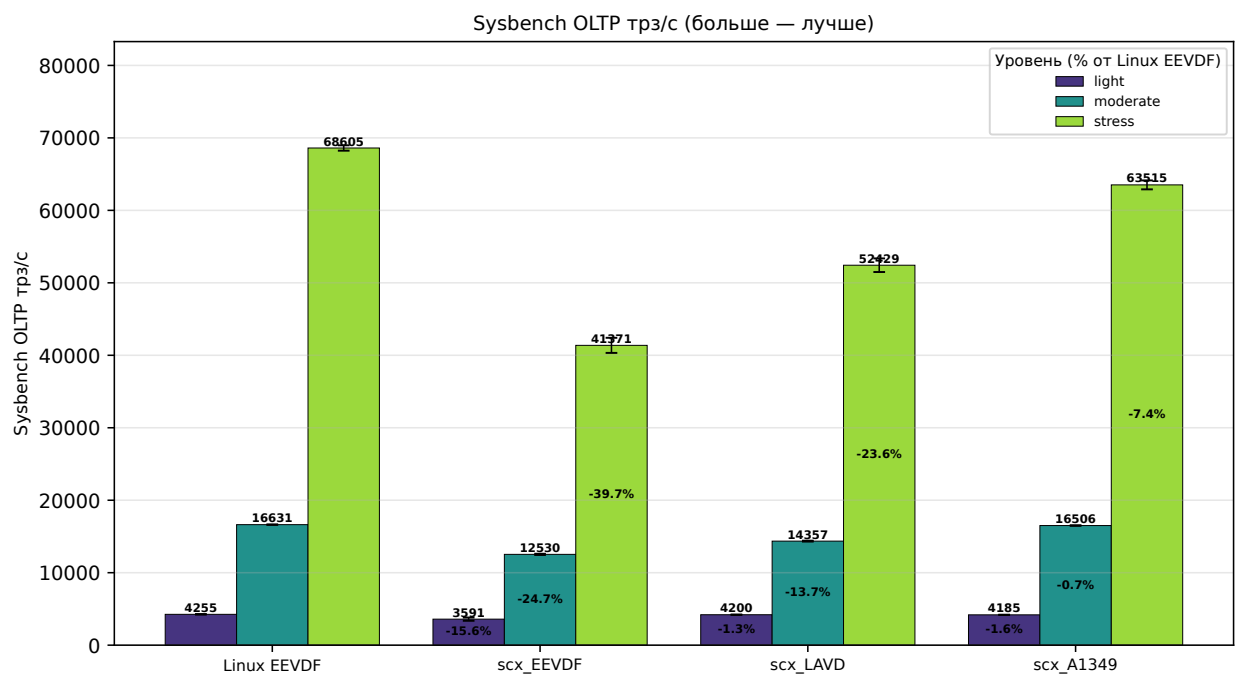


Рис. 5.2: Пропускная способность sysbench (TPS) для четырёх планировщиков по трём режимам нагрузки

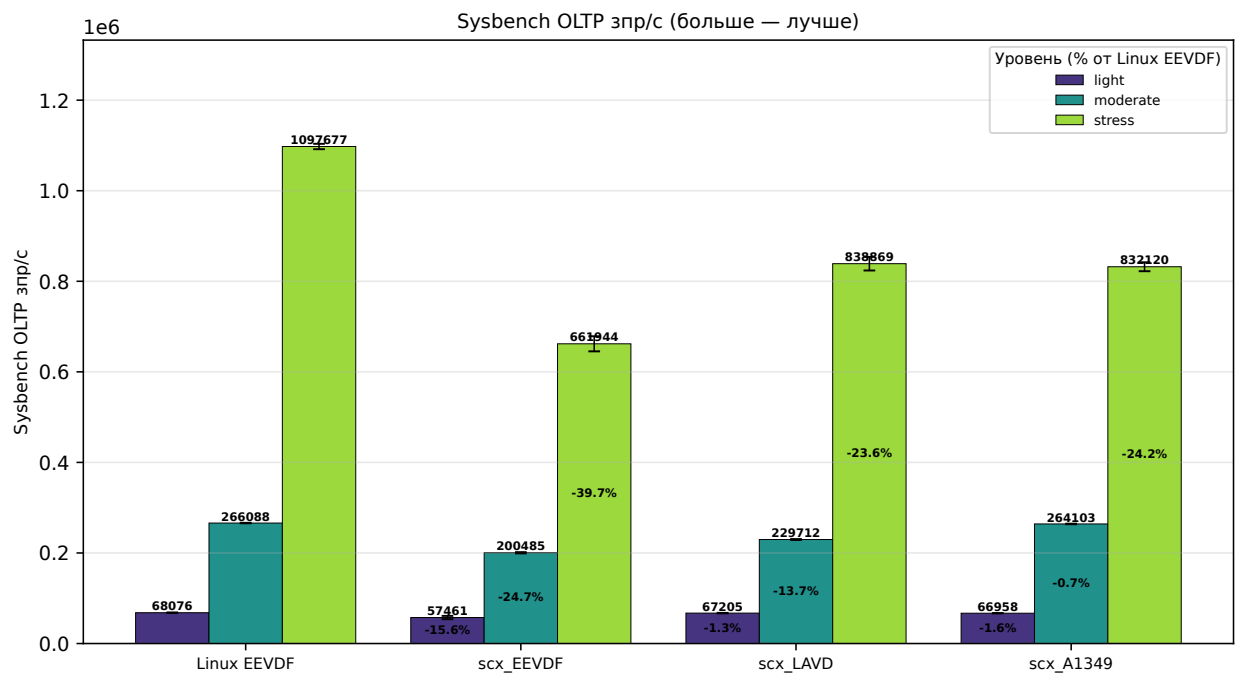


Рис. 5.3: Пропускная способность sysbench (QPS) для четырёх планировщиков по трём режимам нагрузки

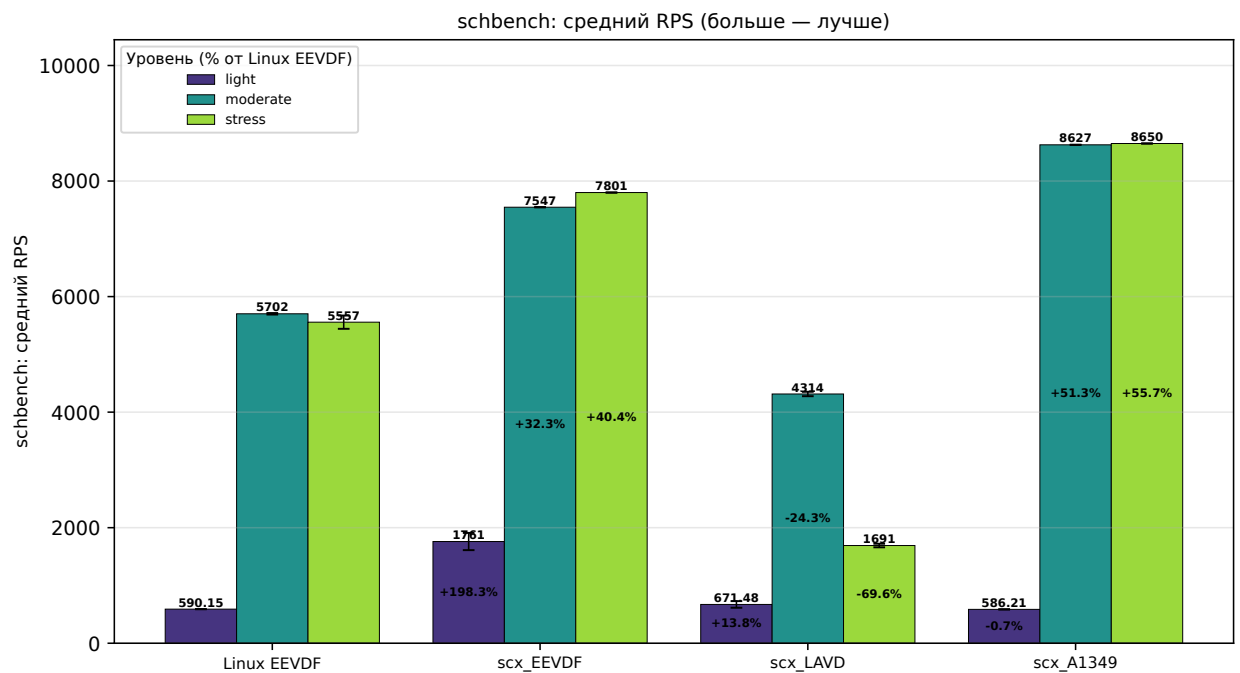


Рис. 5.4: Средняя пропускная способность schbench (запросов в секунду) для четырёх планировщиков по трём режимам нагрузки

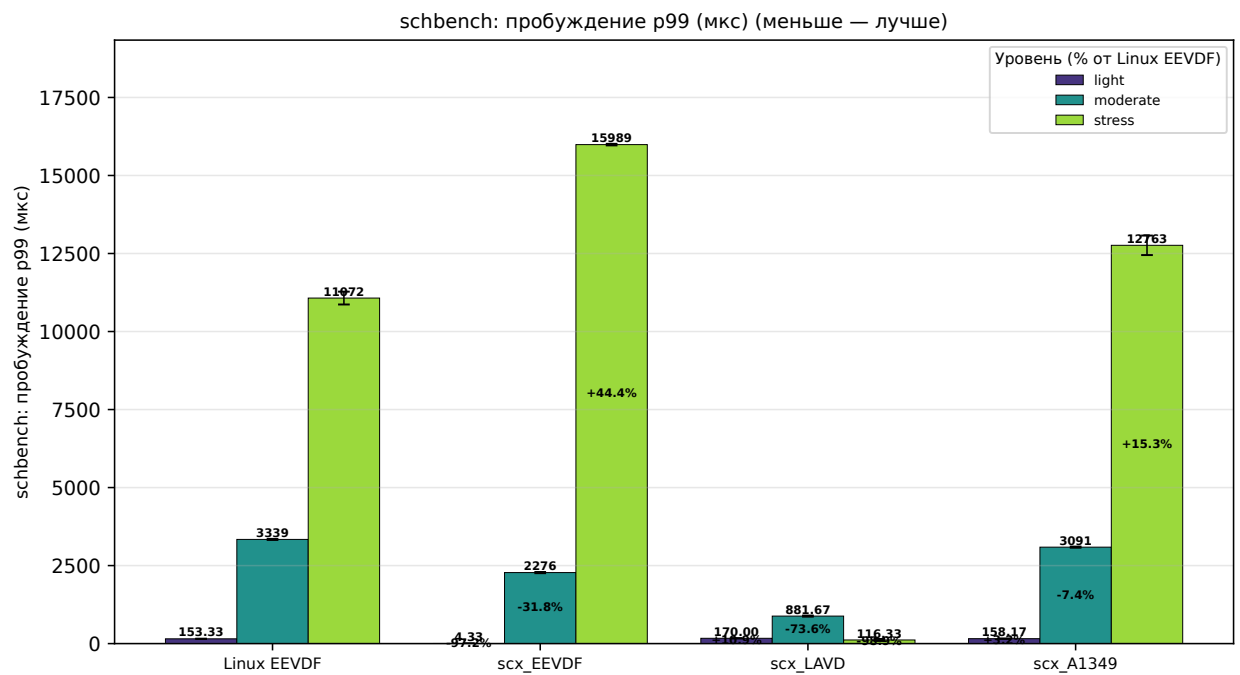


Рис. 5.5: Задержка пробуждения p99 schbench (мкс) для четырёх планировщиков по трём режимам нагрузки

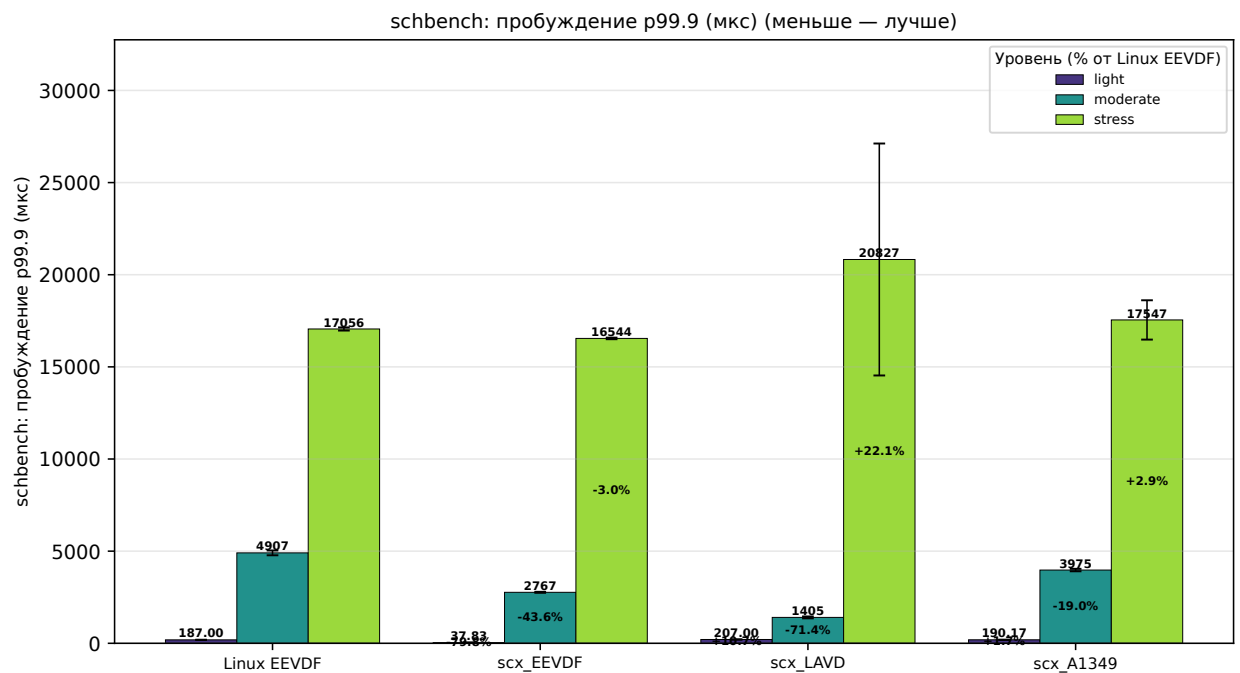


Рис. 5.6: Задержка пробуждения p99.9 schbench (мкс) для четырёх планировщиков по трём режимам нагрузки

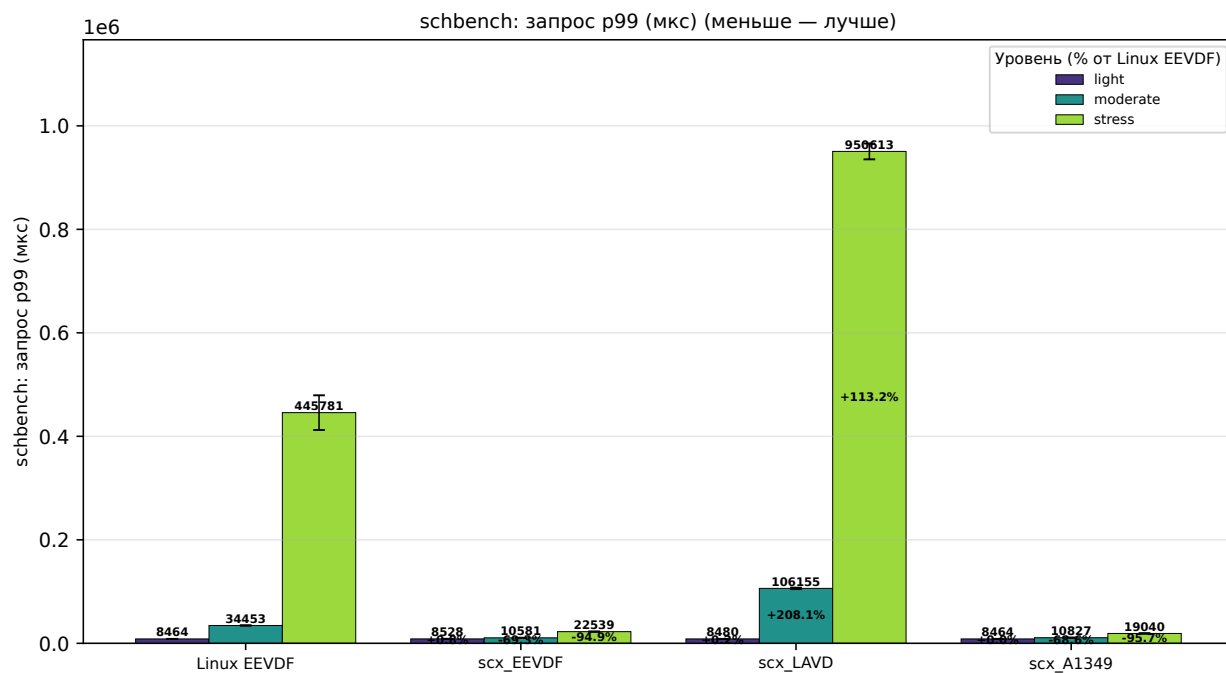


Рис. 5.7: Задержка запроса p99 schbench (мкс) для четырёх планировщиков по трём режимам нагрузки

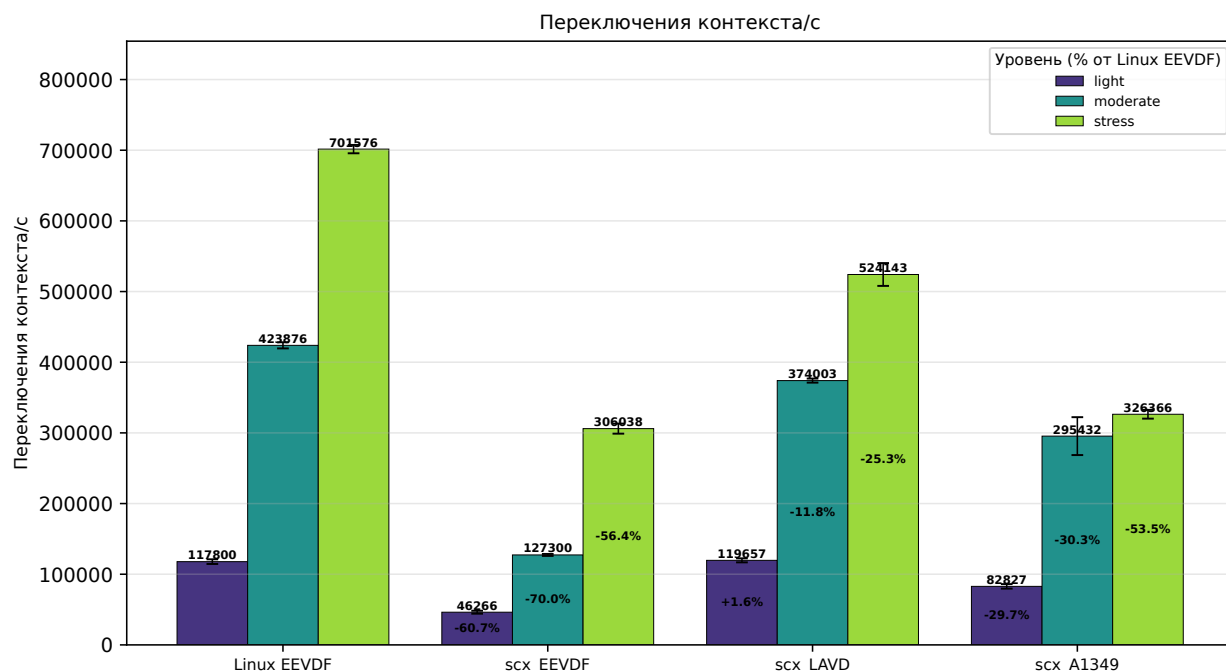


Рис. 5.8: Темп переключений контекста (число в секунду) для четырёх планировщиков по трём режимам нагрузки

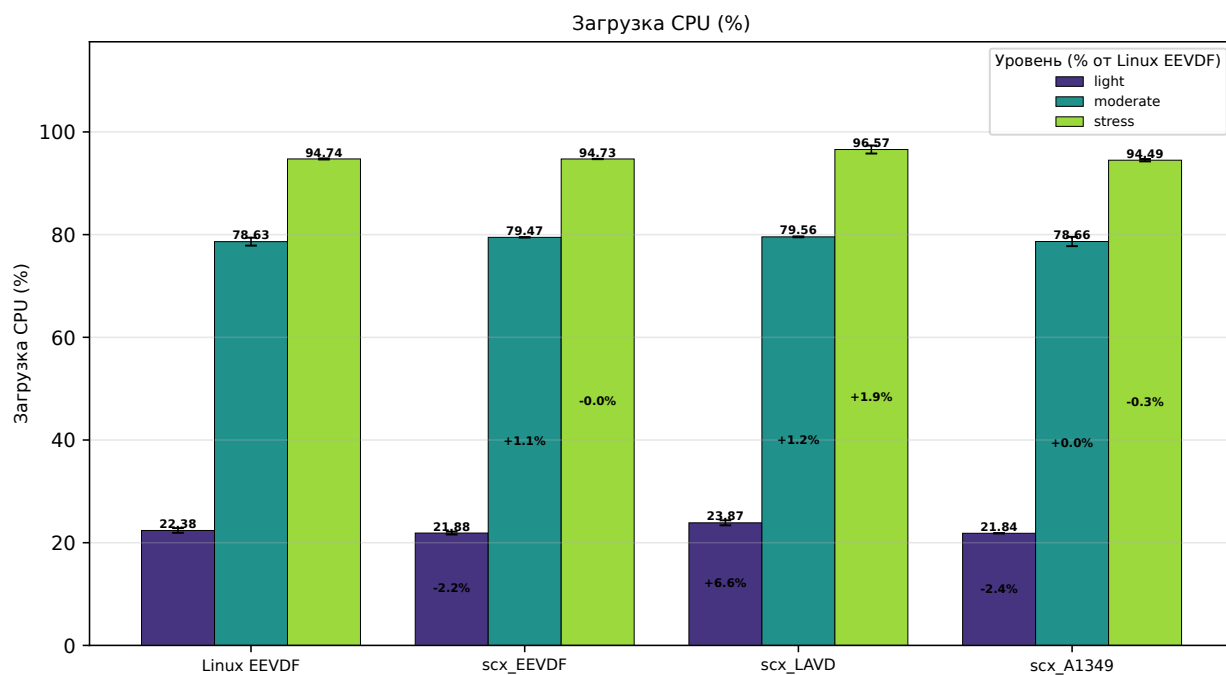


Рис. 5.9: Среднее использование CPU (%) для четырёх планировщиков по трём режимам нагрузки

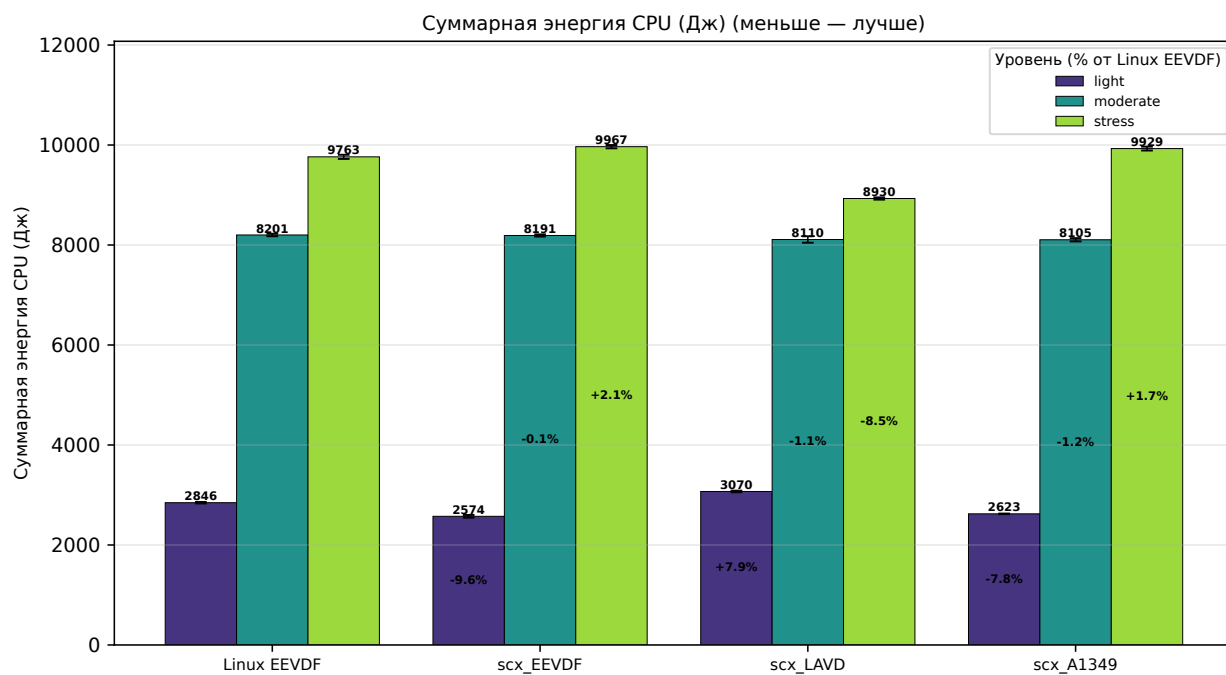


Рис. 5.10: Суммарное энергопотребление (Дж) для четырёх планировщиков по трём режимам нагрузки

Выводы

Проведённый эксперимент позволяет сформулировать основные закономерности поведения планировщика `scx_A1349` на гетерогенной платформе в сравнении со штатным `EEVDF`, реализацией того же алгоритма поверх `sched_ext (scx_EEVDF)` и планировщиком `LAVD`.

Преимущества модели `scx_A1349`. На бенчмарке `hackbench` планировщик `scx_A1349` опережает штатный `EEVDF` на всех трёх уровнях нагрузки: на 4,5 % при лёгкой, на 8,5 % при умеренной и на 1,4 % при стрессовой. При лёгкой и стрессовой нагрузке `scx_A1349` занимает первое место по среднему среди всех четырёх планировщиков, однако в обоих этих режимах его доверительный интервал пересекается с интервалом `scx_EEVDF`, поэтому различие между ними статистически не значимо. Результат объясняется тем, что аукционный механизм направляет короткоживущие задачи `hackbench` преимущественно на Р-ядра, где они завершаются быстрее.

По пропускной способности `schbench` (средний RPS) планировщик `scx_A1349` лидирует при умеренной (8627) и стрессовой (8650) нагрузке: на 51,3 % выше штатного `EEVDF` при умеренной и на 55,7 % при стрессовой нагрузке. При стрессовой нагрузке `scx_A1349` показывает наименьшие значения `p99` и `p99.9` задержки обслуживания запроса (19,04 мс и 24,51 мс соответственно) среди всех планировщиков. Это может объясняться тем, что аукцион направляет задачи, чувствительные к задержке, на Р-ядра, а бюджетные ограничения не дают отдельным задачам надолго блокировать остальные.

Энергопотребление `scx_A1349` оказывается наименьшим или близким к наименьшему: –7,8 % от штатного `EEVDF` при лёгкой нагрузке и –1,2 % при умеренной нагрузке.

Ограничения модели `scx_A1349`. На бенчмарке `sysbench` при стрессовой нагрузке `scx_A1349` отстаёт от штатного `EEVDF` на 7,4 % (63 515 TPS против 68 605 TPS). Аналогичное, но меньшее отставание (1,6 % при лёгкой и 0,8 % при умеренной) сохраняется на других уровнях нагрузки. При этом `scx_A1349` существенно превосходит вторую `sched_ext`-реализацию (`scx_EEVDF`) во всех режимах. Часть отставания от штатного `EEVDF`

обусловлена общими свойствами `sched_ext`: накладными расходами на BPF-вызовы при высоком темпе переключений контекста.

По хвостовой задержке пробуждения при лёгкой нагрузке `scx_A1349` уступает `scx_EEVDF` более чем на порядок (p99 158 мкс против 4 мкс). Поскольку оба планировщика построены на `sched_ext`, различие, по-видимому, обусловлено дополнительной логикой выбора ядра в `scx_A1349` (поиск свободного P-ядра при пробуждении), которая при отсутствии конкуренции увеличивает время пробуждения, не давая выигрыша от назначения на более производительное ядро.

Влияние свойств модели на результаты. Учёт типа ядра при назначении задач, заложенный в модель A1349, наиболее заметен в режимах умеренной и стрессовой нагрузки: `scx_A1349` лидирует по средней пропускной способности `schbench` (8627 и 8650 запросов в секунду соответственно) и сокращает время выполнения `hackbench` относительно штатного `EEVDF` на всех трёх уровнях нагрузки. При стрессовой нагрузке хвостовая задержка обслуживания запроса p99.9 составляет 24,51 мс у `scx_A1349` против $\approx 1,69$ с у штатного `EEVDF` и $\approx 1,81$ с у `LAVD`. Близкое значение `scx_EEVDF` (27,21 мс) показывает, что этот эффект во многом связан с самим `sched_ext`, а не исключительно с аукционным механизмом A1349.

Ограничения эксперимента. Все планировщики тестировались с $n = 6$ запусками на фиксированной длительности. Эксперимент проведён на единственной аппаратной платформе (Intel Arrow Lake-S), и переносимость результатов на архитектуры ARM big.LITTLE и RISC-V требует дополнительной проверки. Кроме того, использованные бенчмарки не покрывают нагрузки реального времени и сценарии с интенсивным обращением к GPU, для которых поведение модели может существенно отличаться.

Заключение

В работе предложен и реализован планировщик для гетерогенных архитектур, проведена его экспериментальная оценка на реальной аппаратной платформе. Выполнен обзор подходов к реализации планировщиков в операционных системах и обоснован выбор фреймворка `sched_ext`.

Построена математическая модель A1349 на основе динамического VCG-механизма, трактующая выбор ядра для задачи как исход аукциона. Эффективная ценность задачи учитывает стоимость кванта целевого ядра, отражая разницу производительности P/E ядер. Аукцион назначает задачу на тот тип ядра, где её эффективная ценность максимальна, VCG-платёж задаёт цену вытеснения конкурента, а бюджетные ограничения предотвращают монополизацию производительных ядер.

Модель и алгоритм EEVDF реализованы средствами `sched_ext`, что позволило сравнить A1349 со штатным EEVDF, реализацией EEVDF в `sched_ext` (`scx_EEVDF`) и LAVD. Подготовлена воспроизводимая методика измерений на основе бенчмарков `hackbench`, `sysbench` и `schbench` при трёх уровнях нагрузки, и проведён эксперимент на платформе Intel Arrow Lake-S с P/E ядрами.

На `hackbench` `scx_A1349` опережает штатный EEVDF на всех уровнях нагрузки: на 4,5 % при лёгкой, на 8,5 % при умеренной и на 1,4 % при стрессовой. На `schbench` при умеренной и стрессовой нагрузке он лидирует по средней пропускной способности (+51,3 % и +55,7 % относительно штатного EEVDF). По хвостовой задержке при стрессовой нагрузке (p99.9) `scx_A1349` опережает штатный EEVDF и LAVD на два порядка (24,51 мс против $\approx 1,69$ с и $\approx 1,81$ с). На `sysbench` все `sched_ext`-планировщики отстают от штатного EEVDF, что указывает на накладные расходы фреймворка.

Эксперимент показал, что учёт гетерогенности на уровне математической модели даёт измеримый выигрыш на нагрузках, где размещение задач по типам ядер влияет на результат.

Перспективным направлением дальнейших исследований является проверка переносимости модели A1349 на процессоры ARM big.LITTLE и гетерогенные конфигурации RISC-V.

Список литературы

- [1] John Lions A Commentary on the Sixth Edition UNIX Operating System The University of New South Wales 1977
- [2] Mike Kravetz, Hubertus Franke, Shailabh Nagar, Rajan Ravindran Enhancing Linux Scheduler Scalability 2001
- [3] Robert Love Linux Kernel Development 2nd ed Novell Press, 2005
- [4] C. S. Wong, I. K. T. Tan, R. D. Kumari, J. W. Lam, W. Fun Fairness and Interactive Performance of O(1) and CFS Linux Kernel Schedulers International Symposium on Information Technology 2008
- [5] Ion Stoica, Hussein Abdel-Wahab Earliest Eligible Virtual Deadline First : A Flexible and Accurate Mechanism for Proportional Share Resource Allocation Old Dominion University 1996
- [6] Greenhalgh P. Big.LITTLE Processing with ARM Cortex-A15 & Cortex-A7 ARM White Paper, 2011
- [7] Rotem E. et al. Alder Lake Architecture 2021 IEEE Hot Chips 33 Symposium (HCS), Palo Alto, CA, USA, 2021, pp. 1–23, doi: 10.1109/HCS52781.2021.9567097
- [8] Conti F., Paulin G., Garofalo A., Rossi D., Pullini A., Benini L. Marsellus: A Heterogeneous RISC-V AI-IoT End-Node SoC with 2-to-8b DNN Acceleration and 30%-Boost Adaptive Body Biasing IEEE Journal of Solid-State Circuits, 2024
- [9] Energy Aware Scheduling [Электронный ресурс]. Linux Kernel Documentation. Режим доступа: <https://www.kernel.org/doc/html/latest/scheduler/sched-energy.html> – Дата обращения 14.11.2025
- [10] Corbet J. The BPF extensible scheduler class LWN.net, 30 November 2022 Режим доступа: <https://lwn.net/Articles/916291/> – Дата обращения 20.11.2025

- [11] Corbet J. Sched_ext at LPC 2024 LWN.net, 2024 Режим доступа: <https://lwn.net/Articles/991205/> – Дата обращения 11.09.2025
- [12] Humphries J. T., Natu N., Chaugule A., Weisse O., Rhoden B., Don J., Rizzo L., Rombakh O., Turner P., Kozyrakis C ghOST: Fast & Flexible User-Space Delegation of Linux Scheduling SOSP 2021
- [13] Tang Q., Gao X., Li G., Lin J Optimizing Linux Scheduling Based on Global Runqueue with SCX IEEE International Conference SMC 2024
- [14] Xu J., Wolff D., Yi H. X., Li J., Roychoudhury A. Concurrency Testing in the Linux Kernel via eBPF arXiv 2025
- [15] Mao J., Ujcich B. E., Ma S Rethinking Provenance Completeness with a Learning-Based Linux Scheduler arXiv 2025
- [16] Wang X., Jia S., Huang Z., Cao J., Song M Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies arXiv 2025
- [17] Galin M., Hedblom A Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads Linköping University 2025
- [18] Van Craeynest K., Jaleel A., Eeckhout L., Narvaez P., Emer J. S Scheduling Heterogeneous Multi-Cores through Performance Impact Estimation (PIE) ISCA, ACM, 2012, pp. 213–224
- [19] Sano R. A Dynamic Mechanism Design for Scheduling with Different Use Lengths Institute of Economic Research, Kyoto University, 2013
- [20] Dobzinski S., Lavi R., Nisan N. Multi-unit Auctions with Budget Limits // Games and Economic Behavior. 2012. Vol. 74, No. 2. P. 486–503. (Extended abstract: FOCS, 2008. P. 260–269.)